

[Deep Learning Framework]

Deep Neural Networks and Convolution Neural Networks

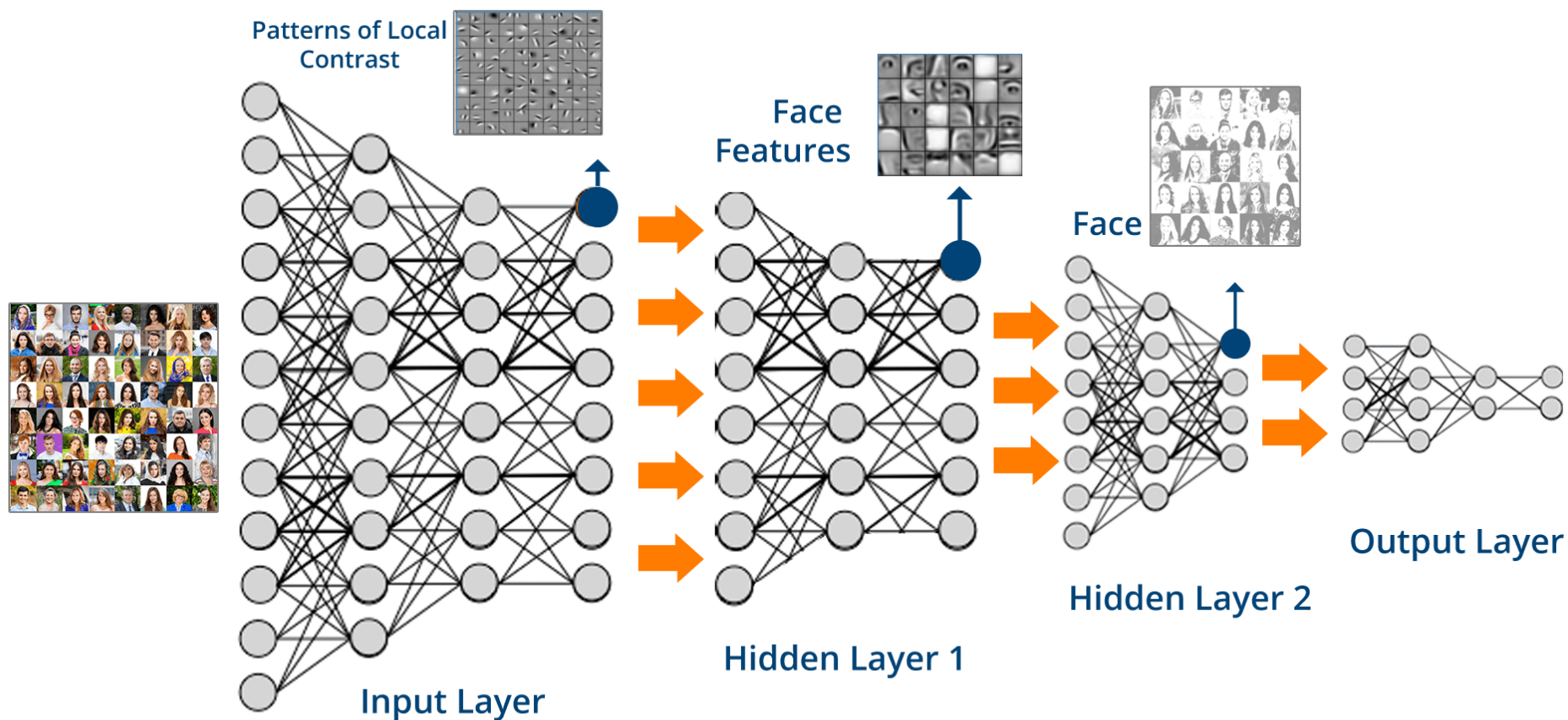
박성호

컴퓨터공학부

한국공학대학교

Deep Neural Network (DNN, 심층 신경망)

- Deep Neural Network (DNN)
 - 여러 층으로 이루어진 복잡한 구조의 인공신경망



Deep Neural Network 역사

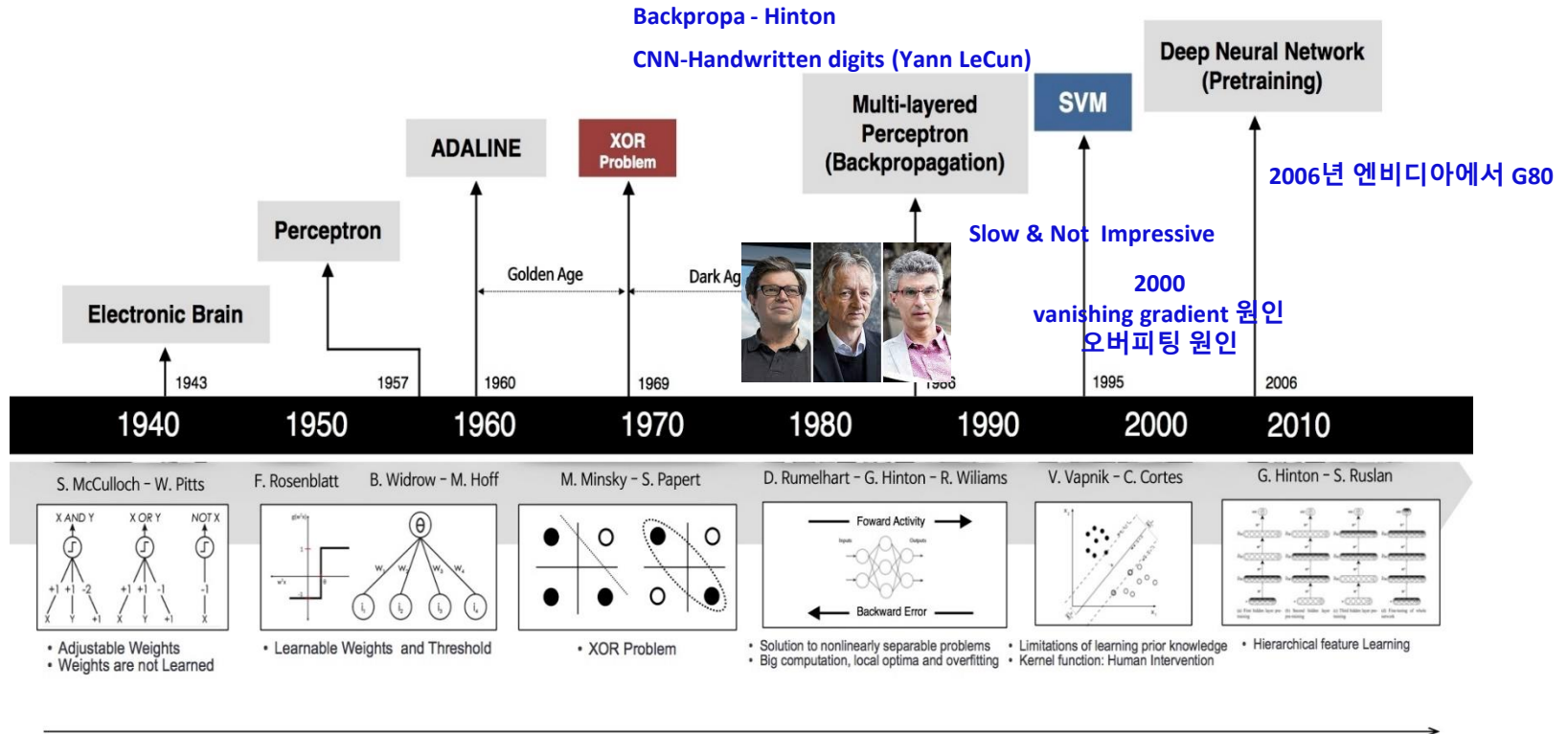
- 2006년 Geoffrey Hinton에 의해 많은 층(layer)을 학습할 수 있게 됨
- 그 이전에는 2개 이상의 층을 학습하기 거의 불가능했음

이후 2011 Adagrad 개발

이후 2013 ReLu 적용

이후 2014 Adam 개발

이후 2014 Dropout 개발



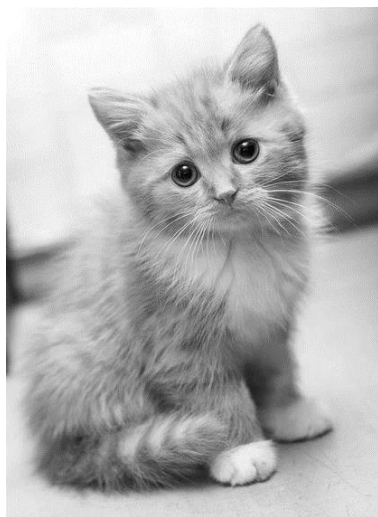
Brief history of neural network

<https://www.packtpub.com/product/deep-learning-essentials/9781785880360>

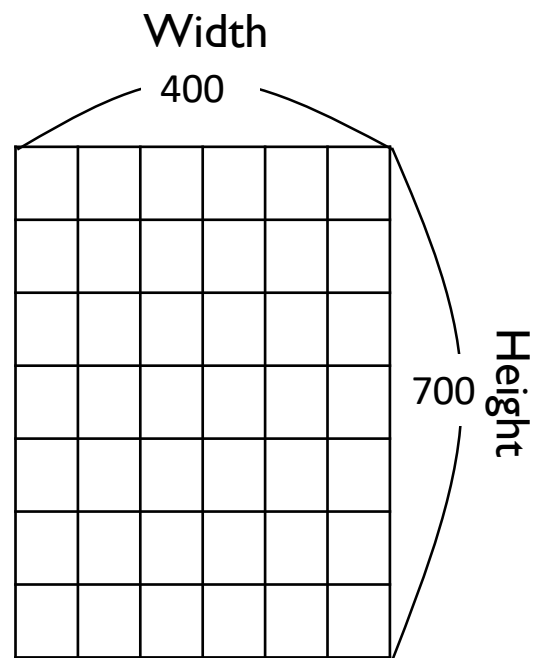
Convolution Neural Networks (CNN)

Image, Vision

- 흑백 이미지는 2차원의 행렬로 표현됨 (Width, Height), 행렬 원소 $\rightarrow$ 픽셀
- 행렬은 흑백의 톤(짙고 옅은 정도)을 실수 값으로 표현
- 8 bit: 1~256 / 16 bit: 1~65,536



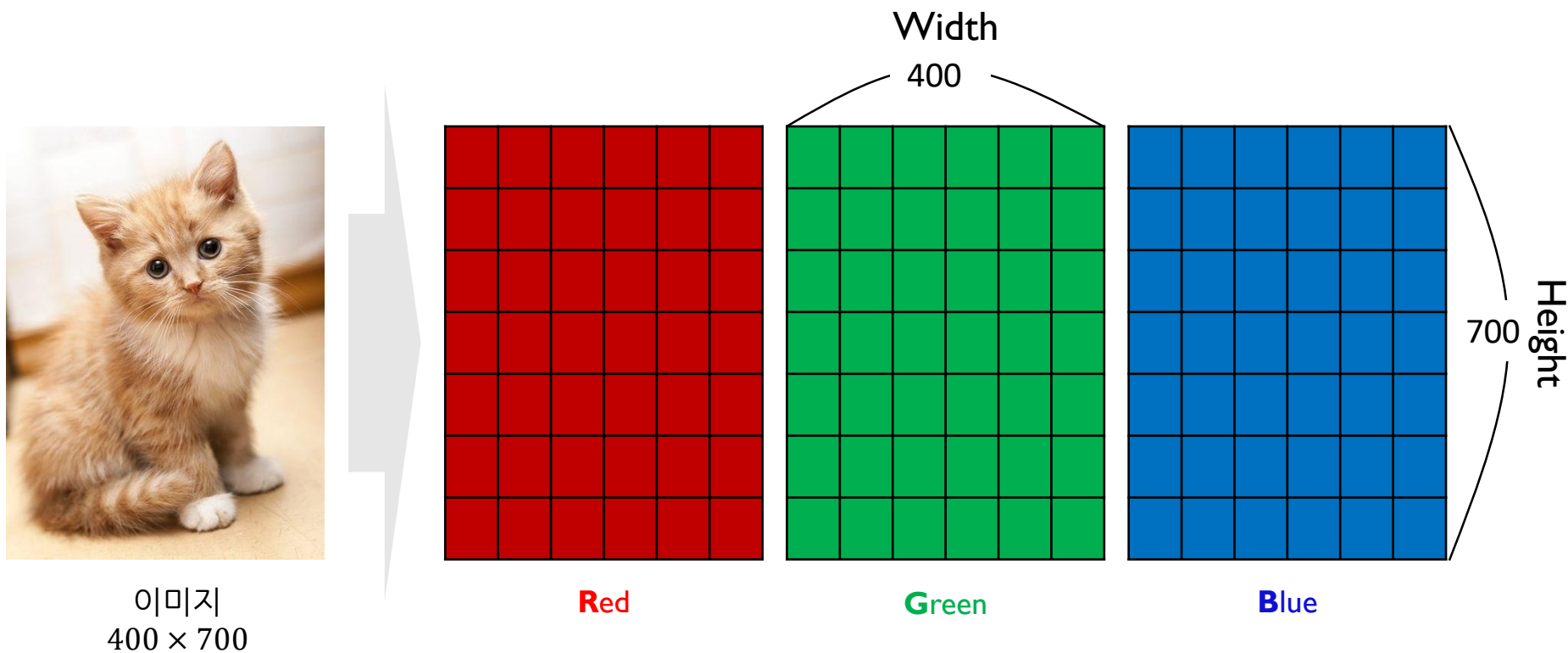
이미지
 400×700



Convolution Neural Networks (CNN)

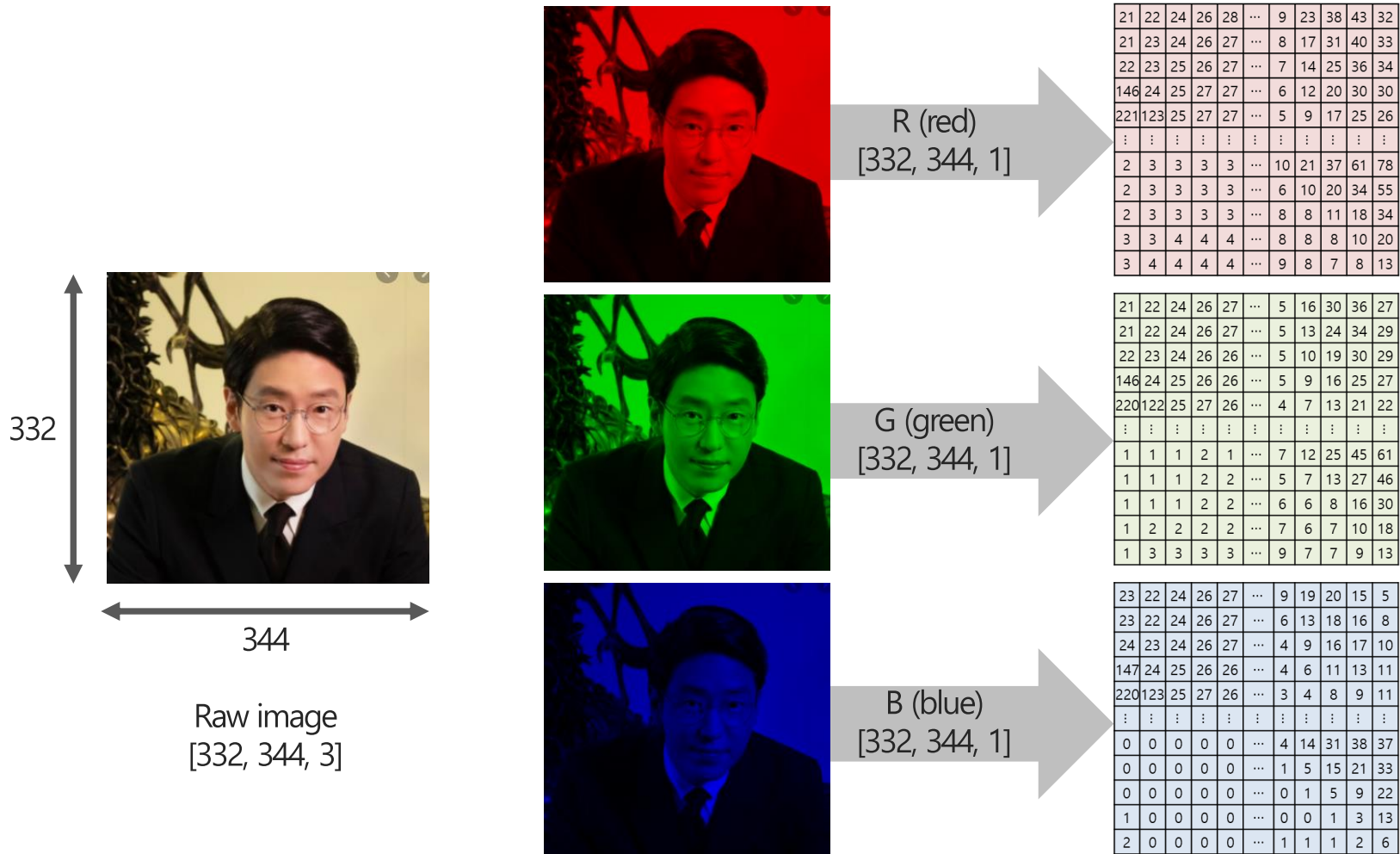
Image, Vision

- 컬러 이미지는 3차원의 Tensor로 표현됨: Width, Height, RGB (Red, Green, Blue)
- 각각의 행렬은 색의 톤을 실수 값으로 표현
- 8 bit: 1~256 / 16 bit: 1~65,536



Convolution Neural Networks (CNN)

Image, Vision



Convolution Neural Networks (CNN)

Image, Vision



Raw image
[332, 344, 3]



R (red)
[332, 344, 1]



G (green)
[332, 344, 1]



B (blue)
[332, 344, 1]

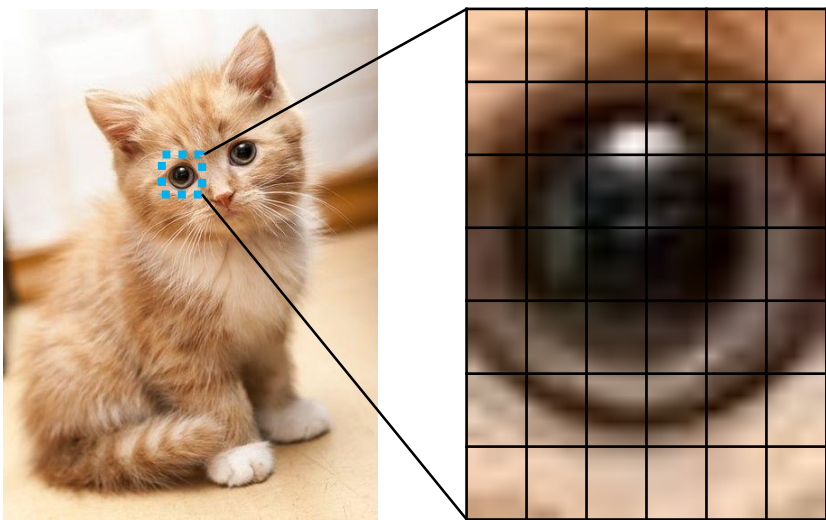
121	122	116	113	109	108	111	112	115	117	118	119	115	107	101	99	98	96	97	97	96	94	94	97	100	105	115	126	135	144	153	161		
136	129	125	121	116	115	119	125	128	133	134	132	131	133	131	129	128	124	119	111	111	110	107	106	107	108	111	119	128	137	143	151	159	
141	137	135	130	126	131	135	139	144	147	149	148	148	148	150	148	143	138	132	126	121	119	117	115	118	120	121	128	135	141	147	151	159	
145	144	142	140	142	148	152	153	156	157	158	159	159	160	160	155	155	151	147	137	130	126	123	122	127	130	132	138	142	147	151	155	159	
146	147	144	143	148	152	155	156	156	157	161	162	163	164	164	162	160	157	148	140	137	131	127	132	137	140	143	150	154	157	160	161		
140	141	137	135	138	144	149	150	152	154	157	163	166	167	170	174	178	184	188	187	186	179	173	170	164	158	157	155	154	158	162	165	170	
134	132	130	127	129	135	141	144	146	151	156	161	166	170	174	178	184	188	187	186	179	173	170	164	158	157	155	154	158	162	165	170		
139	133	127	125	127	130	136	138	143	146	150	156	159	163	170	171	168	172	179	186	184	185	183	178	173	172	170	168	170	173	175	179	176	
139	131	124	123	124	127	133	139	142	146	152	159	163	168	176	179	180	182	186	186	183	185	181	179	180	185	183	179	180	181	183	185	188	
132	124	113	119	127	135	145	157	166	174	179	184	188	191	195	200	206	213	220	221	219	207	196	188	184	186	189	187	198	187	191	193		
146	126	110	119	130	138	145	151	159	161	162	155	150	142	136	137	142	153	167	165	202	216	219	209	200	198	195	189	194	192	188	193	187	
152	129	126	135	141	145	146	144	139	130	110	103	89	79	74	75	81	82	110	126	142	161	184	193	194	195	198	197	196	194	192	195	189	
118	122	131	137	139	144	127	121	110	95	83	75	64	48	41	39	41	44	71	105	125	119	138	153	170	182	189	199	206	203	191	182	192	
116	119	126	127	123	117	112	105	86	82	89	89	71	46	35	27	25	31	56	110	189	90	111	128	136	153	173	188	202	207	199	190	196	
117	119	119	116	102	85	73	105	134	126	100	65	42	27	20	25	38	64	79	108	106	107	116	116	138	169	194	208	207	180	185	179		
120	122	123	124	125	120	108	100	83	108	133	137	137	79	48	34	26	28	35	49	85	151	165	127	104	107	114	139	175	200	207	203	194	174
129	131	133	131	127	131	132	134	135	133	154	142	80	59	49	42	41	48	63	108	175	200	172	122	98	107	113	148	185	201	201	198	174	
138	139	143	139	134	139	151	160	165	169	170	165	148	110	78	65	60	61	71	82	146	191	206	199	163	152	83	94	122	168	198	186	177	
154	155	154	153	149	146	154	162	169	175	184	182	190	176	147	124	113	111	120	149	186	204	212	206	191	156	123	107	114	158	196	195	162	179
166	167	168	167	164	158	160	166	169	172	180	189	194	199	196	193	190	196	196	196	205	211	208	201	196	192	181	187	156	171	193	182	187	168
173	175	177	177	170	166	166	171	175	182	186	192	196	201	206	209	210	216	214	213	210	208	205	207	208	210	205	193	179	184	188	153	148	
179	181	182	183	183	178	172	170	173	178	185	193	196	198	203	207	210	213	218	216	216	216	212	205	200	196	197	192	179	167	162	159	142	
184	184	186	187	185	182	179	177	174	176	183	189	193	196	202	207	212	214	216	216	216	214	213	204	190	176	167	165	169	165	150	141	140	
187	189	191	192	191	188	184	179	174	175	179	183	187	191	196	200	205	207	208	207	206	203	204	201	193	179	162	151	157	171	157	152	156	
191	193	195	196	197	196	194	190	185	182	181	182	186	192	195	198	200	196	195	194	195	194	185	170	155	148	161	162	138	165	165	146	146	
194	195	196	201	202	202	201	199	192	187	182	176	173	174	176	177	180	184	186	190	195	197	197	193	180	164	159	173	150	130	141	140	139	
198	196	199	202	205	208	207	204	198	193	188	180	174	175	178	181	185	191	197	200	206	211	210	207	196	179	173	164	162	131	134	143	143	
195	198	201	204	208	210	209	205	201	197	195	192	193	195	197	201	205	208	210	211	218	219	205	191	186	199	192	174	180	151	148	151	146	
194	195	195	196	201	206	211	213	214	211	210	210	207	208	207	207	208	210	214	215	218	222	223	223	221	198	195	213	219	214	184	154	147	
193	196	197	198	199	202	206	208	210	210	211	211	210	208	209	209	211	212	214	215	217	220	223	223	224	202	203	219	222	216	183	151	150	
198	198	201	201	200	201	203	206	209	210	212	211	211	212	209	209	211	213	215	216	217	220	223	224	222	212	202	207	219	217	200	174	176	176
178	187	200	207	208	206	206	207	208	209	210	213	213	213	214	216	217	218	220	224	225	223	221	202	208	216	223	188	178	185	185	185		
183	180	200	214	218	219	217	217	216	216	219	219	219	220	221	223	224	225	227	226	218	205	198	197	170	154	180	188	184	172	172	172	172	
192	184	185	203	213	216	220	222	222	221	221	223	224	224	226	227	227	228	228	228	222	211	200	189	182	181	183	187	180	174	168	168	168	
193	186	188	198	206	213	218	222	221	222	223	224	225	225	227	227	227	228	228	228	225	210	190	186	183	184	187	183	175	169	171	171	171	
191	195	197	192	198	205	211	217	219	222	224	226	228	230	227	228	229	228	230	231	229	228	199	188	152	172	203	208	199	162	172	170	175	

Convolution Neural Networks (CNN)

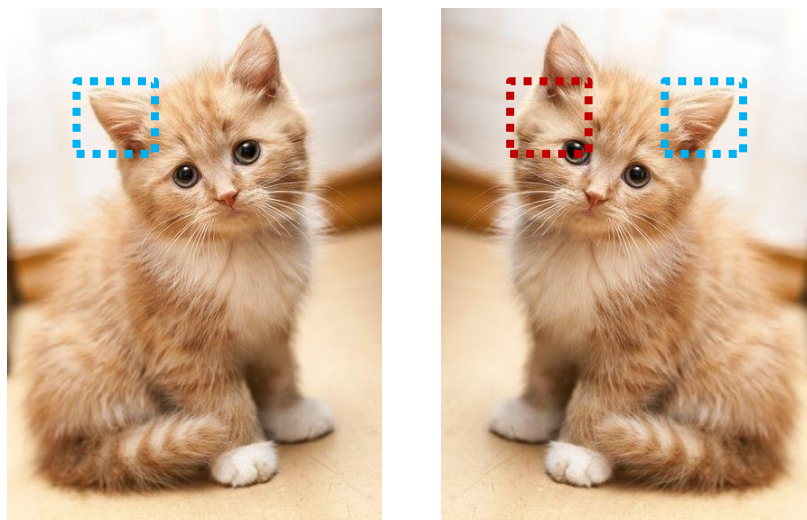
Image, Vision

- 이미지의 경우 인접 변수(픽셀)간 높은 상관관계를 가짐 (spatially-local correlation)
- 이미지의 부분적 특성 (e.g., 눈, 귀)은 고정된 위치에 등장하지 않음 (invariant feature)
- 데이터의 특성을 반영한 모델을 구성하면 성능이 좋아지지 않을까?

Spatially-Local Correlation



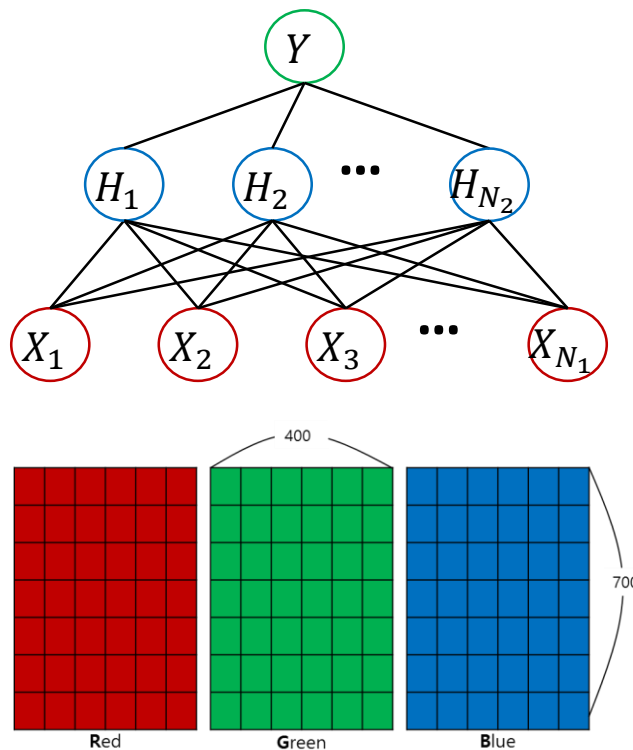
Invariant Feature



Convolution Neural Networks (CNN)

Image, Vision

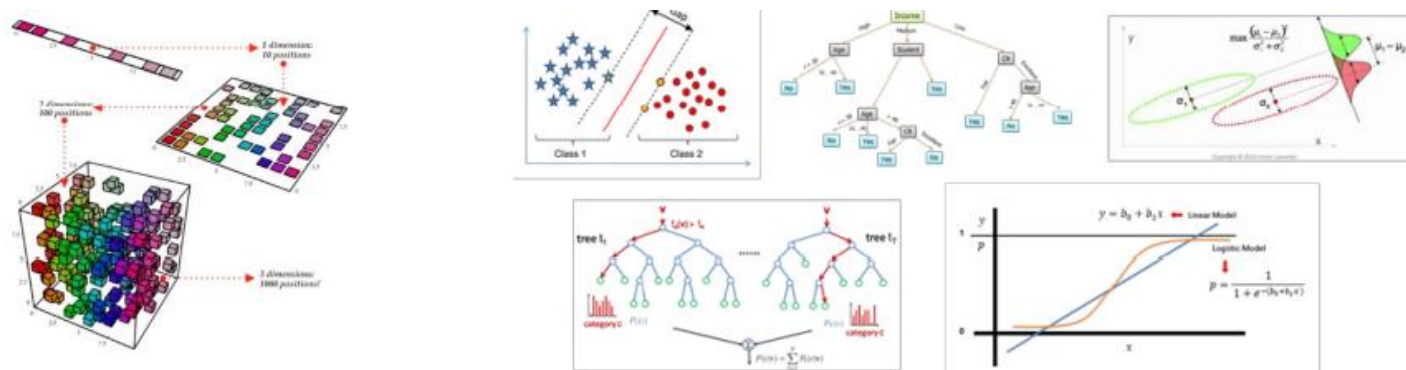
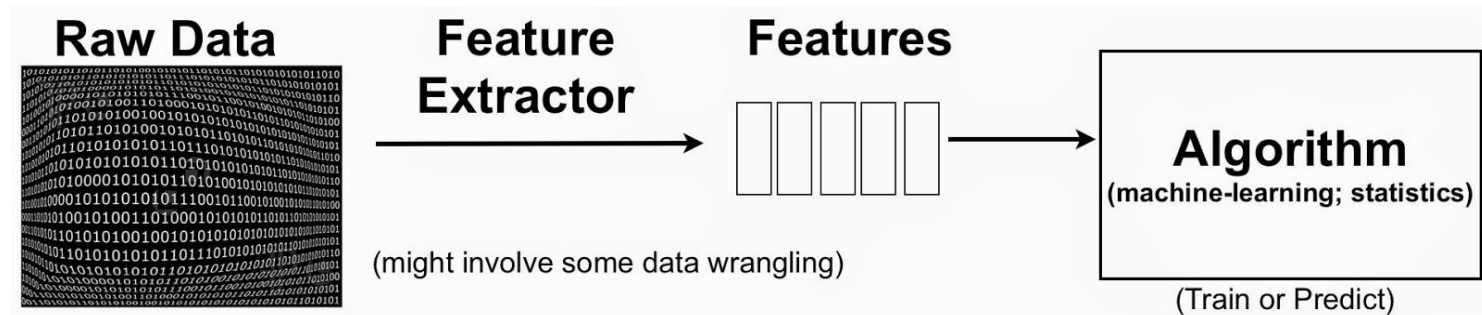
- 각각의 픽셀을 하나의 변수로 간주하여 모델링
- $N_1 = 400(\text{width}) \times 700(\text{height}) \times 3(\text{RGB}) = 840,000$ 개
- N_2 의 크기도 변수의 수에 맞춰서 정함 $\rightarrow (840,000 \times N_2)$ 파라미터 $\rightarrow$ Input layer와 hidden layer 사이에 너무 많은 가중치 $\rightarrow$ **학습이 어렵다!**
- **Spatially-Local Correlation & Invariant Feature x**



Convolution Neural Networks (CNN)

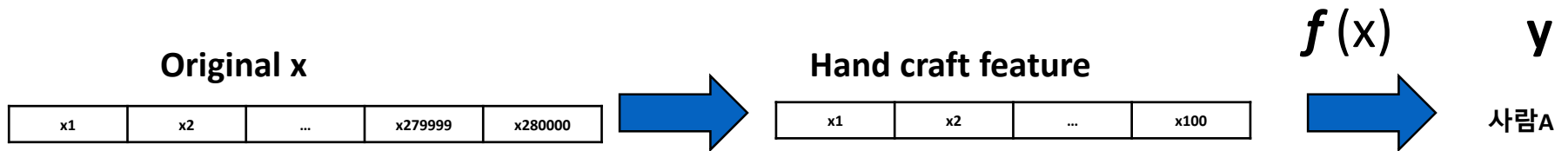
Image, Vision

- 변수의 수를 줄이기 위해 low level 변수 (픽셀)를 조합하여 보다 적은 수의 변수를 생성
- Feature engineering: 도메인 지식, 기계학습, transform기법을 이용하여 feature 생성



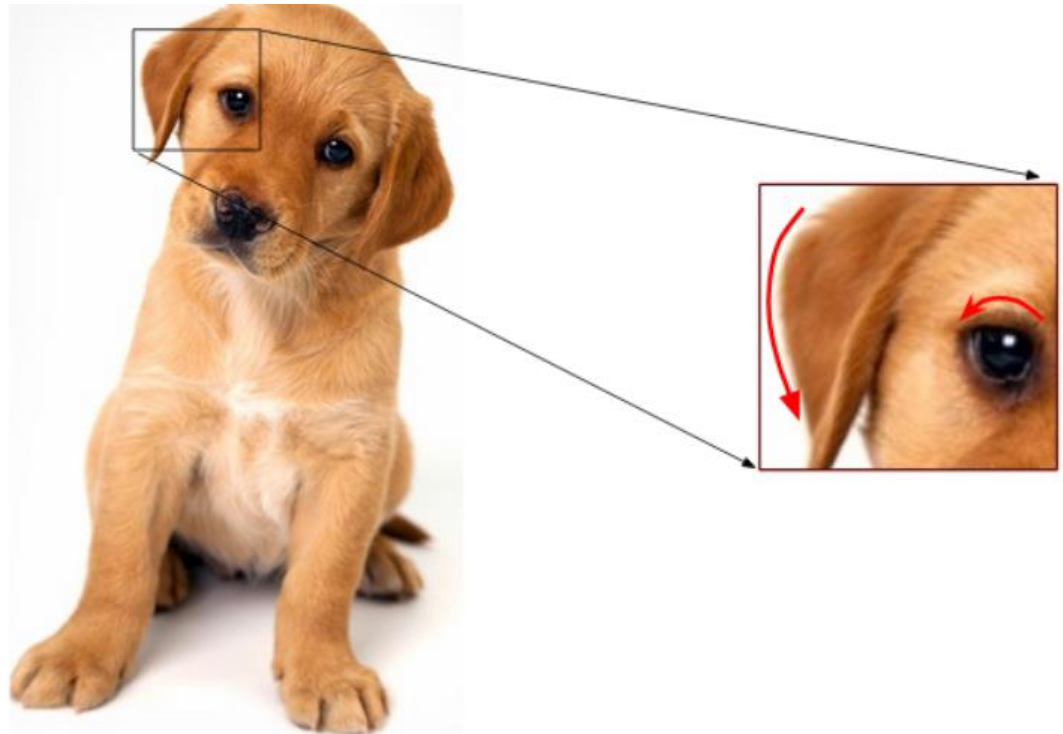
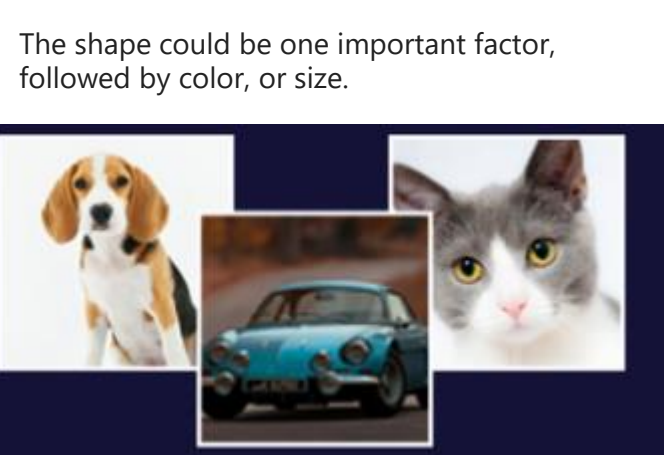
Deep Learning

Feature engineering



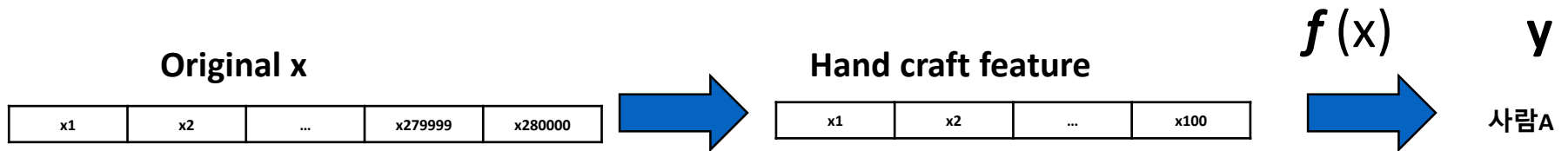
Extracting Edge Features:

Edge is basically where there is a sharp change in color.



Deep Learning

Feature engineering



Extracting Edge Features:

Edge is basically where there is a sharp change in color.

To identify if a pixel is an edge or not, we will simply subtract the values on either side of the pixel.

121	10	78	96	125
48	152	68	125	111
145	78	85	89	65
154	214	56	200	66
214	87	45	102	45

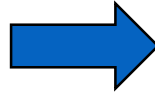
121	10	78	96	125
48	152	68	125	111
145	78	85	89	65
154	214	56	200	66
214	87	45	102	45

Deep Learning

Feature engineering

Original x

x1	x2	...	x279999	x280000
----	----	-----	---------	---------



Hand craft feature

x1	x2	...	x100
----	----	-----	------

$f(x)$



y

사람A

Extracting Edge Features:

Edge is basically where there is a sharp change in color.

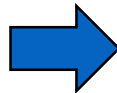
To identify if a pixel is an edge or not, we will simply subtract the values on either side of the pixel.

-1	0	1
-1	0	1
-1	0	1

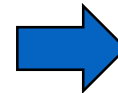
Prewitt Kernel
X Direction

-1	-1	-1
0	0	0
1	1	1

Prewitt Kernel
Y Direction



121	10	78	96	125
48	152	68	125	111
145	78	85	89	65
154	214	56	200	66
214	87	45	102	45



121	10	78	96	125
48	152	68	125	111
145	78	85	89	65
154	214	56	200	66
214	87	45	102	45

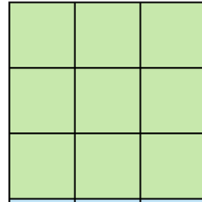
152	68	125
78	85	89
214	56	200

-1	0	1
-1	0	1
-1	0	1

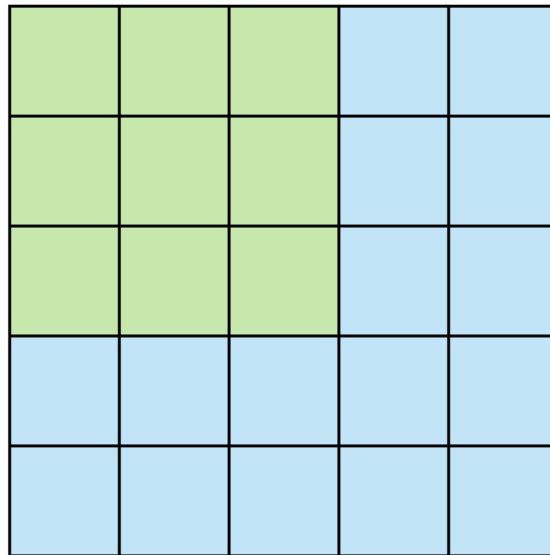
Convolution Neural Networks (CNN)

Image Convolution

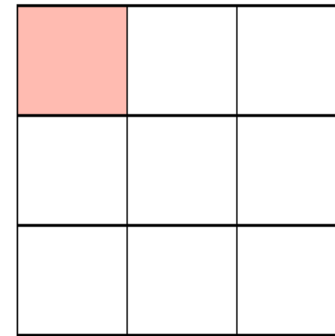
- Convolution with stride 1



Filter (필터) or Kernel (커널)

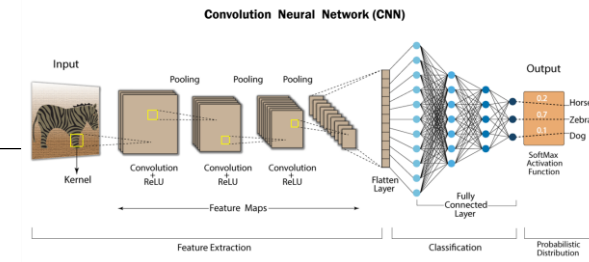


Stride 1



Feature Map

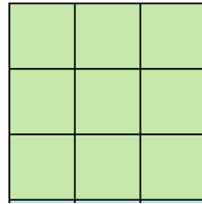
- Stride: 필터의 적용 위치 간격



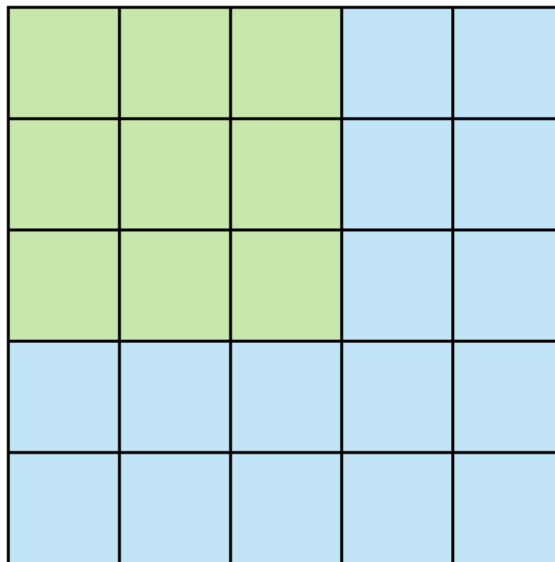
Convolution Neural Networks (CNN)

Image Convolution

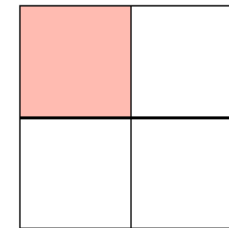
- Convolution with stride 2



Filter (필터)

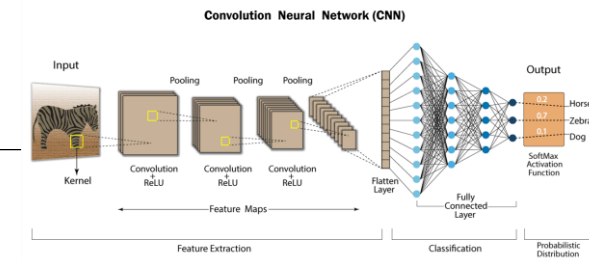


Stride 2



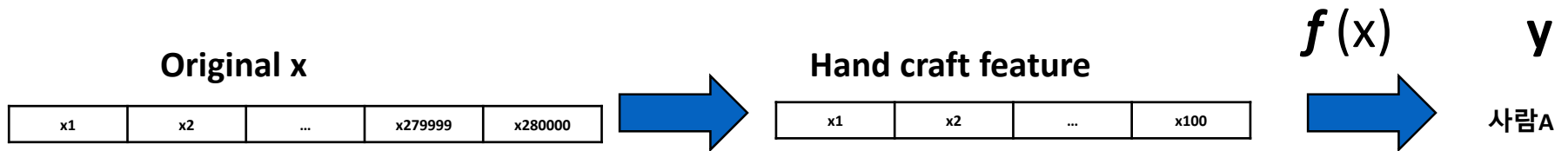
Feature Map

- Large Stride: 이미지의 특징을 놓칠 가능성 증가 → 궁극적으로는 성능 악화



Deep Learning

Feature engineering



Extracting Edge Features:

Edge is basically where there is a sharp change in color.

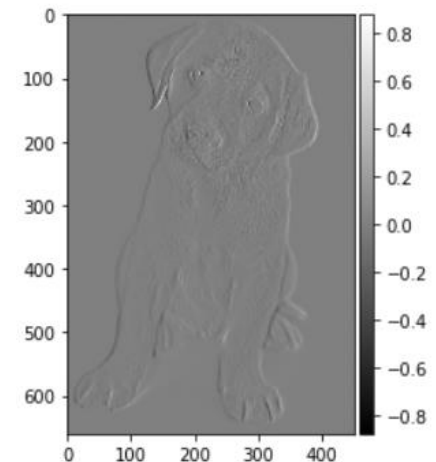
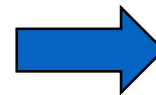
To identify if a pixel is an edge or not, we will simply subtract the values on either side of the pixel.

-1	0	1
-1	0	1
-1	0	1

Prewitt Kernel
X Direction

-1	-1	-1
0	0	0
1	1	1

Prewitt Kernel
Y Direction



Convolution Neural Networks (CNN)

- Neural Computation, Vol. I, pp.541-551, 1989
- First CNN introduced in 1989 by Yann Lecun



Handwritten Digit Recognition with a Back-Propagation Network

Y. Le Cun, B. Boser, J. S. Denker, D. Henderson,
R. E. Howard, W. Hubbard, and L. D. Jackel
AT&T Bell Laboratories, Holmdel, N. J. 07733

ABSTRACT

We present an application of back-propagation networks to handwritten digit recognition. Minimal preprocessing of the data was required, but architecture of the network was highly constrained and specifically designed for the task. The input of the network consists of normalized images of isolated digits. The method has 1% error rate and about a 9% reject rate on zipcode digits provided by the U.S. Postal Service.

1 INTRODUCTION

The main point of this paper is to show that large back-propagation (BP) networks can be applied to real image-recognition problems without a large, complex preprocessing stage requiring detailed engineering. Unlike most previous work on the subject (Denker et al., 1989), the learning network is directly fed with images, rather than feature vectors, thus demonstrating the ability of BP networks to deal with large amounts of low level information.

Previous work performed on simple digit images (Le Cun, 1989) showed that the architecture of the network strongly influences the network's generalization ability. Good generalization can only be obtained by designing a network architecture that contains a certain amount of *a priori* knowledge about the problem. The basic design principle is to minimize the number of free parameters that must be determined by the learning algorithm, without overly reducing the computational power of the network. This principle increases the probability of correct generalization because

of 10 units: one per class. When a pattern belonging to class i is presented, the desired output is $+1$ for the i th output unit, and -1 for the other output units.

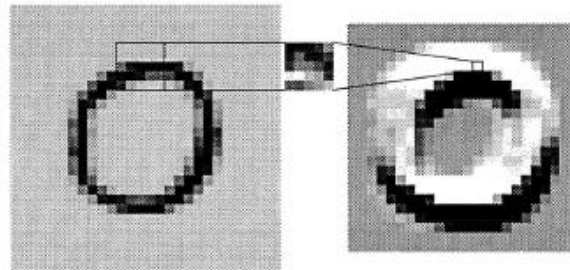
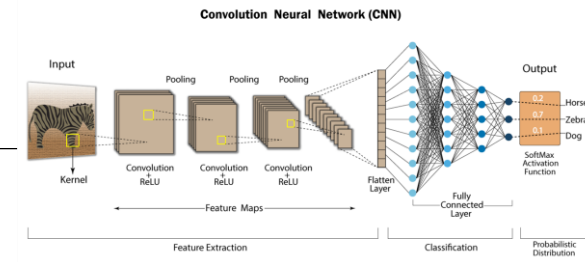


Figure 3: Input image (left), weight vector (center), and resulting feature map (right). The feature map is obtained by scanning the input image with a single neuron that has a local receptive field, as indicated. White represents -1 , black represents $+1$.

A fully connected network with enough discriminative power for the task would have far too many parameters to be able to generalize correctly. Therefore a restricted connection-scheme must be devised, guided by our prior knowledge about shape recognition. There are well-known advantages to performing shape recognition by detecting and combining local features. We have required our network to do this by constraining the connections in the first few layers to be local. In addition, if a feature detector is useful on one part of the image, it is likely to be useful on other parts of the image as well. One reason for this is that the salient features of a distorted character might be displaced slightly from their position in a typical character. One solution to this problem is to scan the input image with a single neuron that has a local receptive field, and store the states of this neuron in corresponding locations in a layer called a *feature map* (see figure 3). This operation is equivalent to a convolution with a small size kernel, followed by a squashing function. The process can be performed in parallel by implementing the feature map as a plane of neurons whose weight vectors are constrained to be equal. That is, units in a feature map are constrained to perform the same operation on different parts of the image. An interesting side-effect of this *weight sharing* technique, already described in (Rumelhart, Hinton and Williams, 1986), is to reduce the number of free parameters by a large amount, since a large number of units share the same weights. In addition, a certain level of shift invariance is present in the system: shifting the input will shift the result on the feature map, but will leave it unchanged otherwise. In practice, it will be necessary to have multiple feature maps, extracting different features from the same image.

Convolution Neural Networks (CNN)

Image Convolution



1	0	1	0	0
1	0	1	1	0
1	0	1	1	1
0	1	0	1	0
1	0	1	0	0

Image

1	0	1
0	1	0
1	0	1

Filter

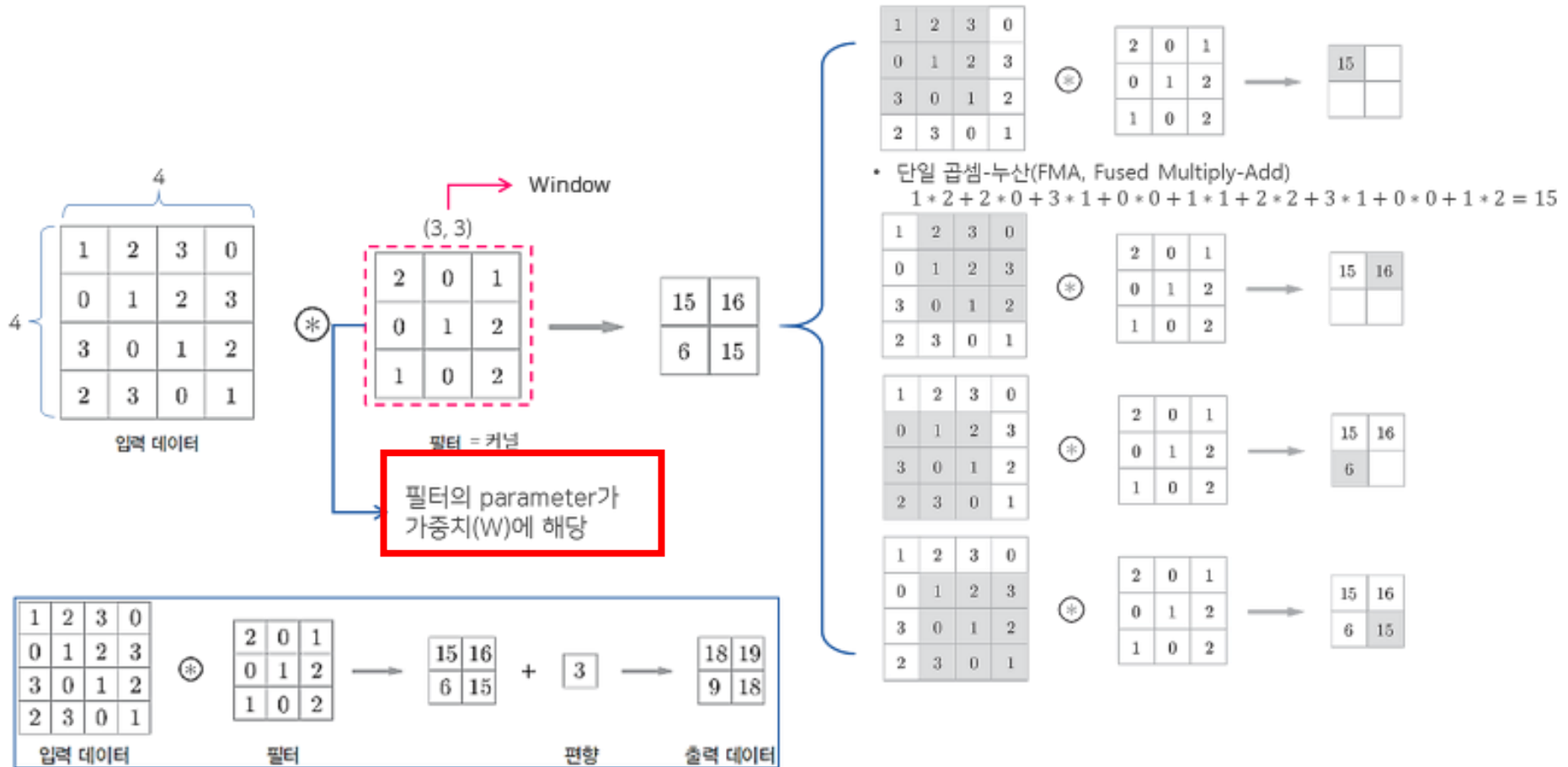
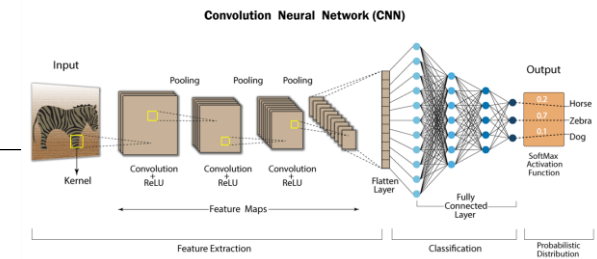
4	3	4
2	4	3
2	3	4

Feature Map

Convolution: slide over the image spatially (preserving 2D or 3D structure), computing dot product

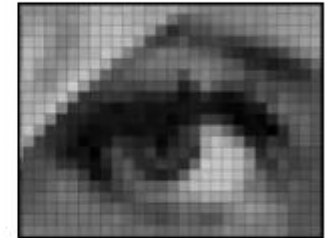
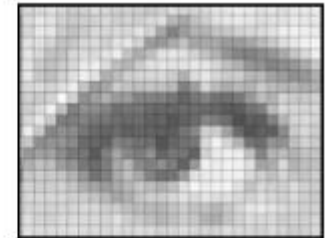
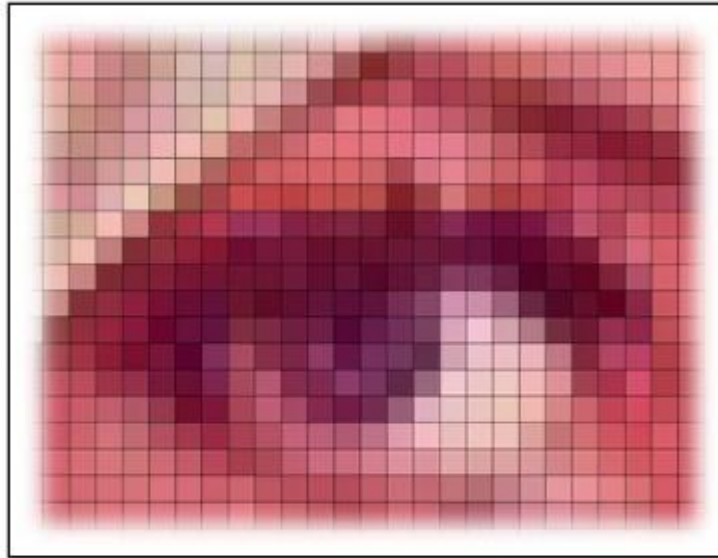
Convolution Neural Networks (CNN)

Image Convolution

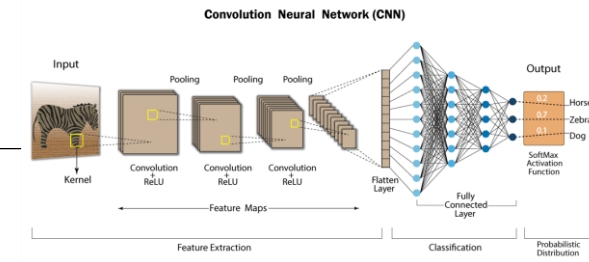
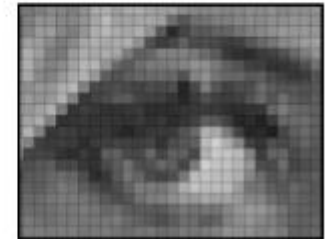


Convolution Neural Networks (CNN)

Image Convolution

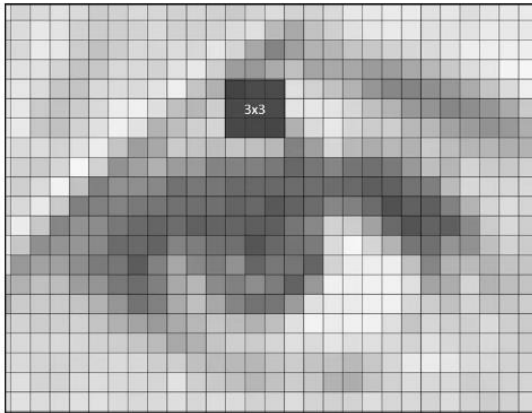


Split by channel
(red, green, blue)



Convolution Neural Networks (CNN)

Image Convolution



These kernel weights, and the bias term, are learned during training

$$\begin{matrix} & \begin{matrix} -0.2 & 0.0 & 0.5 \\ 1.0 & 0.3 & -0.6 \\ 0.0 & 0.0 & 0.8 \end{matrix} \\ * & \end{matrix}$$

Kernel: 3x3

The kernel slides around the image. At each position, the kernel weights are multiplied by the image values and added, plus a bias term

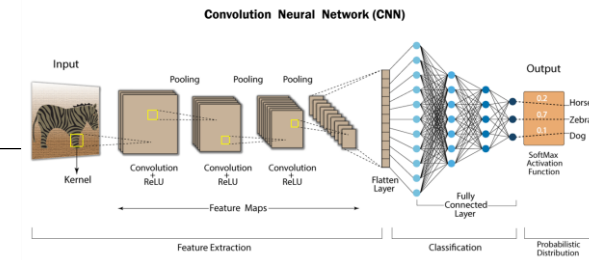
$$s = W_{11}pX_{11} + \dots + W_{33}pX_{33} + b$$

For each region that the kernel covers, one number is computed

-0.2	0.0	0.5
1.0	0.3	-0.6
0.0	0.0	0.8

0.0	0.0	0.0
0.8	-0.5	0.8
0.0	-0.2	0.0

0.4	0.2	-0.2
-0.8	0.0	0.8
0.0	-0.5	0.2

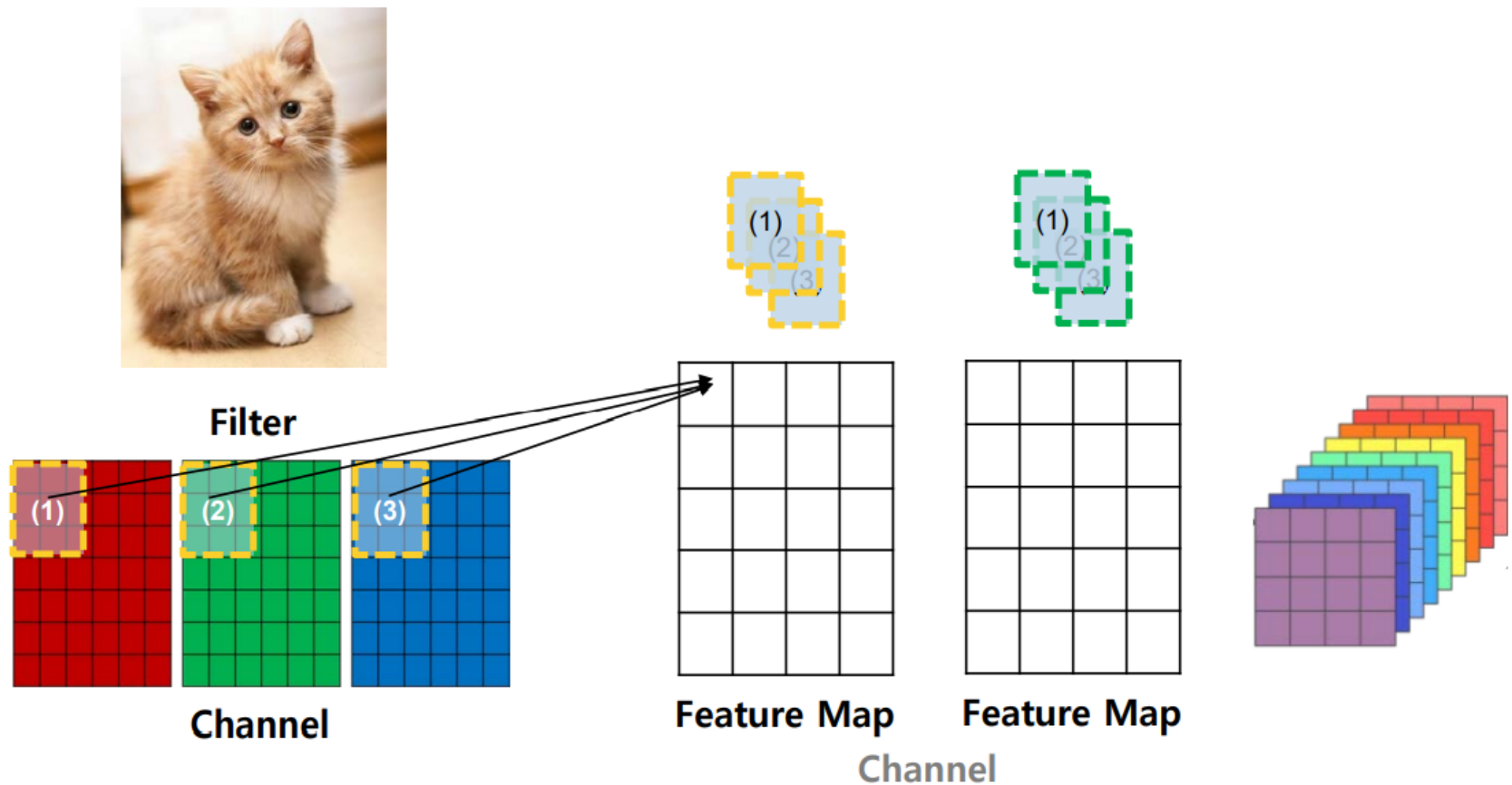


You can see how the image transforms differently depending on the weights. The rightmost image highlights the edges, which is often useful for identifying objects. The stride helps you understand how the kernel slides across the image. The following diagram is an example of a 1 x 1 stride:

Convolution Neural Networks (CNN)

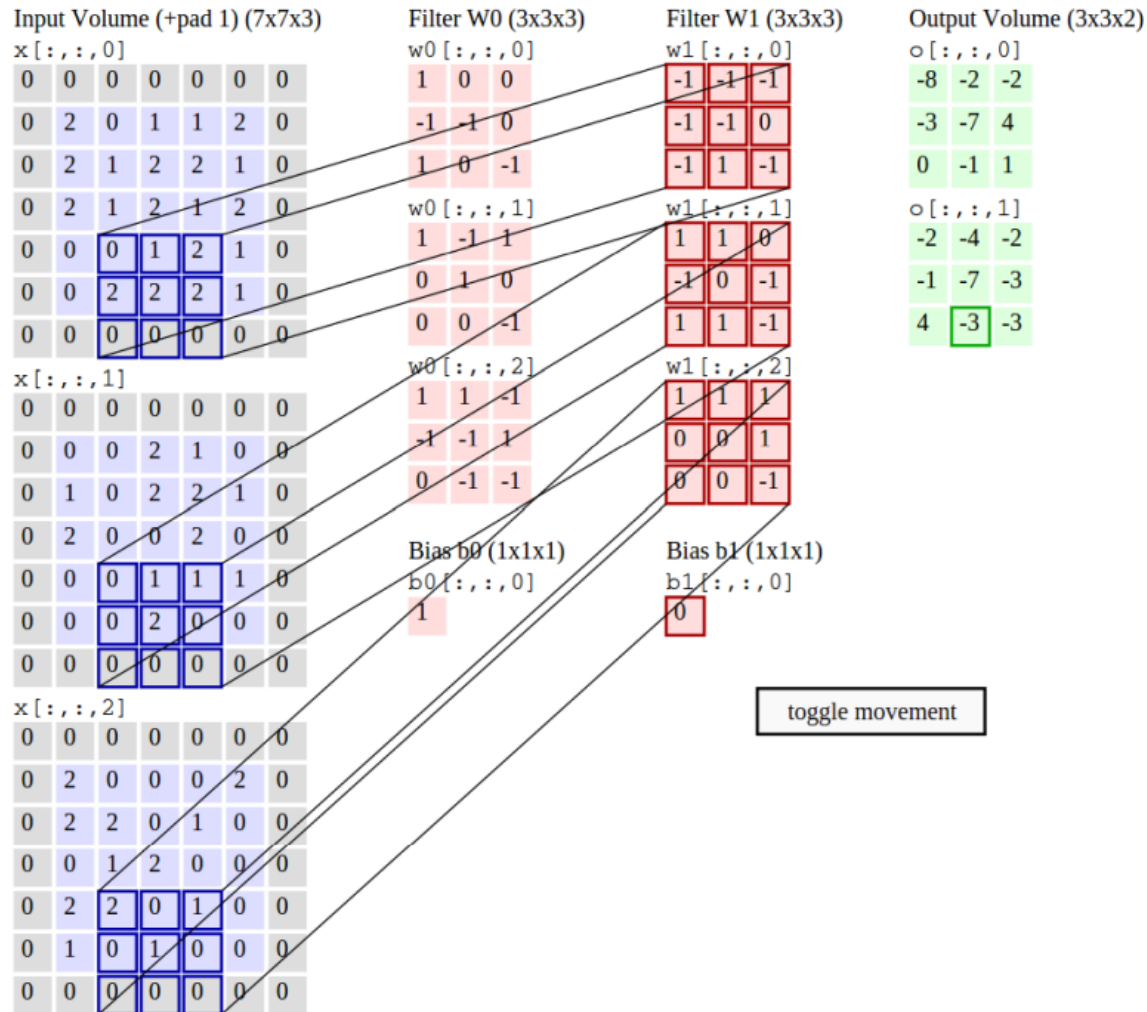
Image Convolution

- ❖ 모든 채널로 부터 얻은 결과값을 합하여 activation 함수에 적용
- ❖ Filter의 수 만큼 feature map이 생성

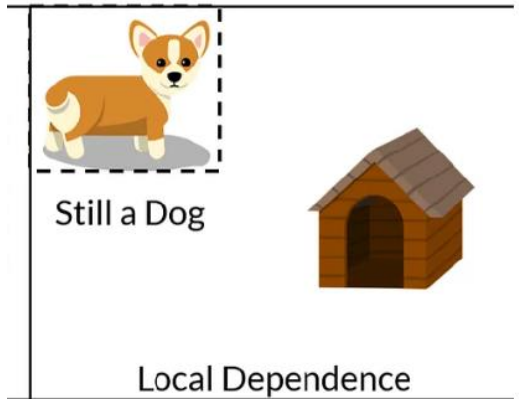
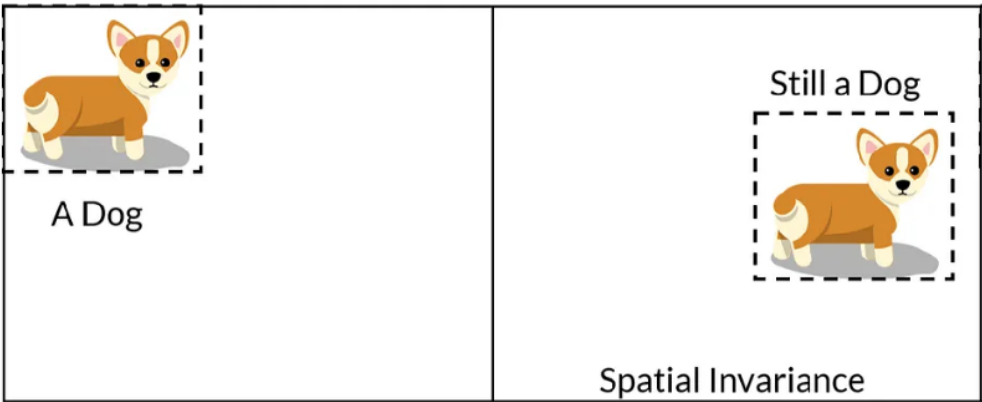


Convolution Neural Networks (CNN)

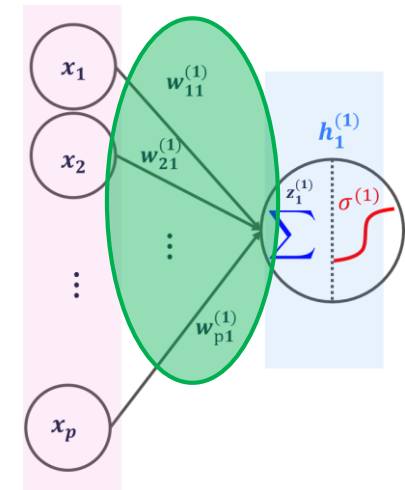
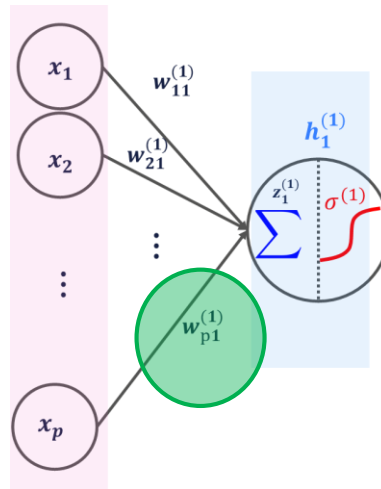
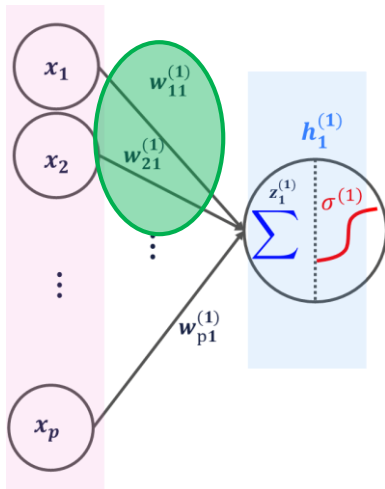
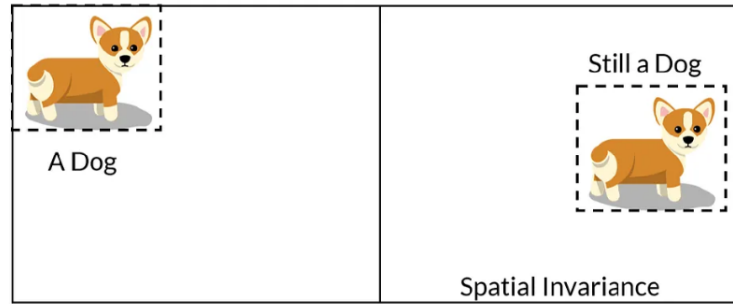
Image Convolution



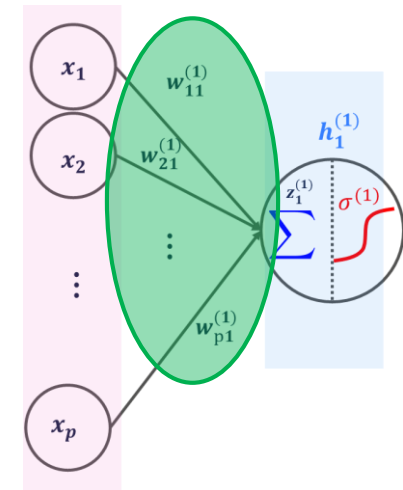
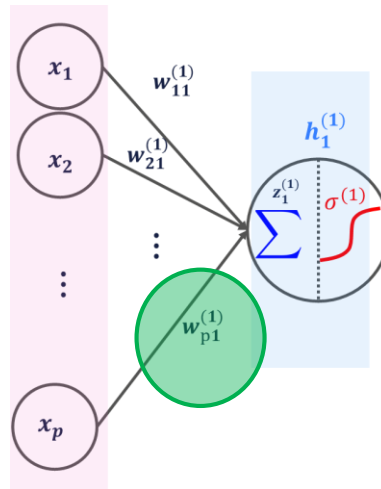
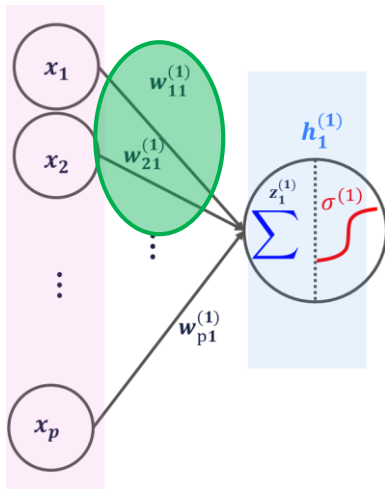
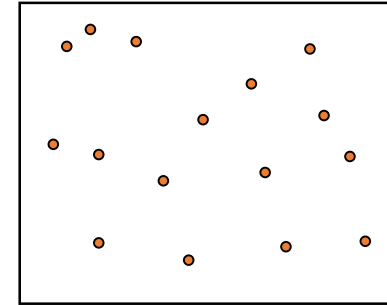
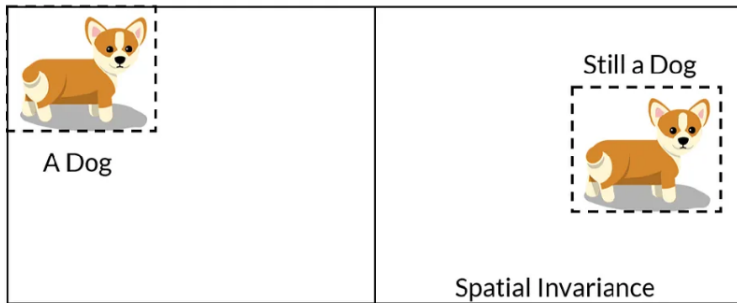
Convolution Neural Networks (CNN)



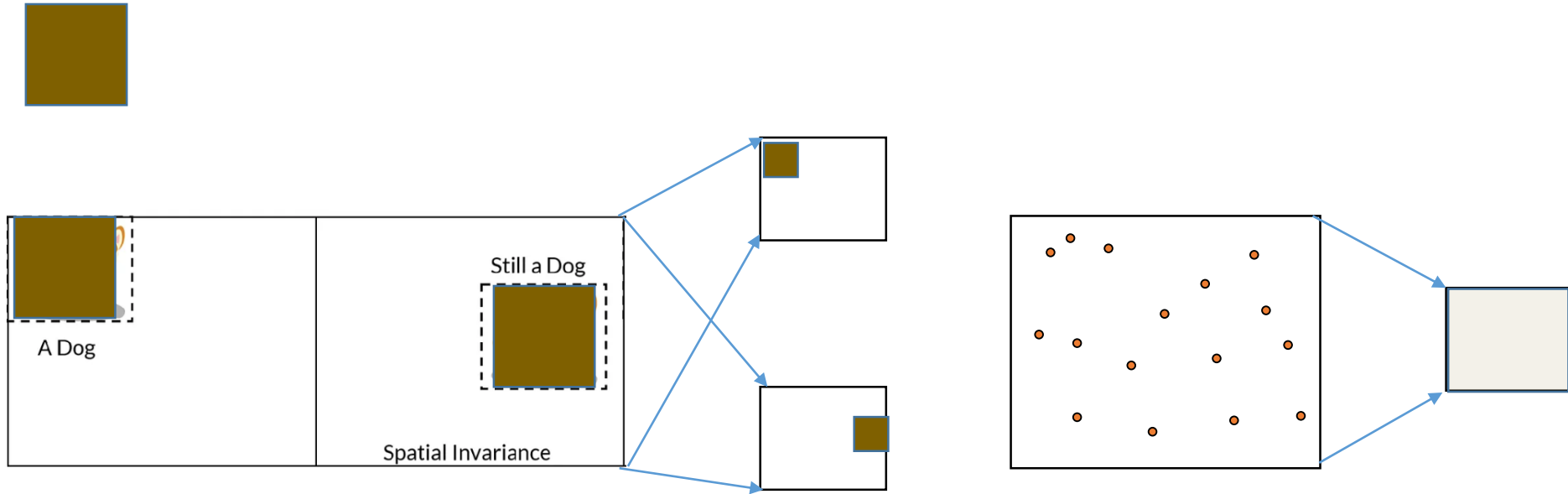
Convolution Neural Networks (CNN)



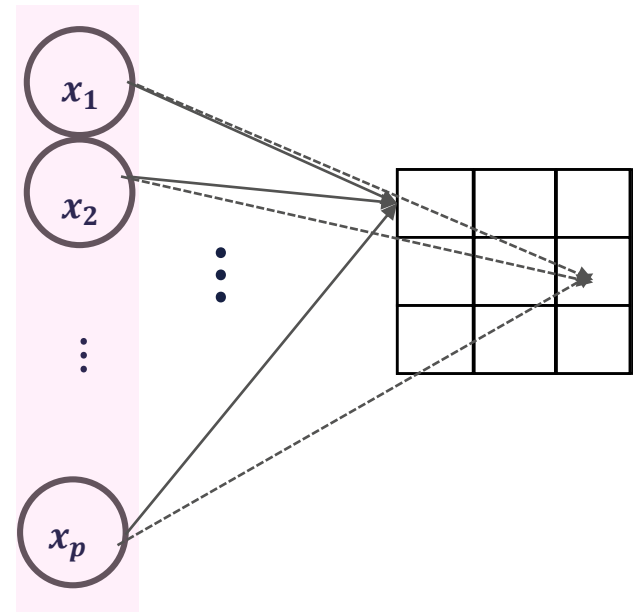
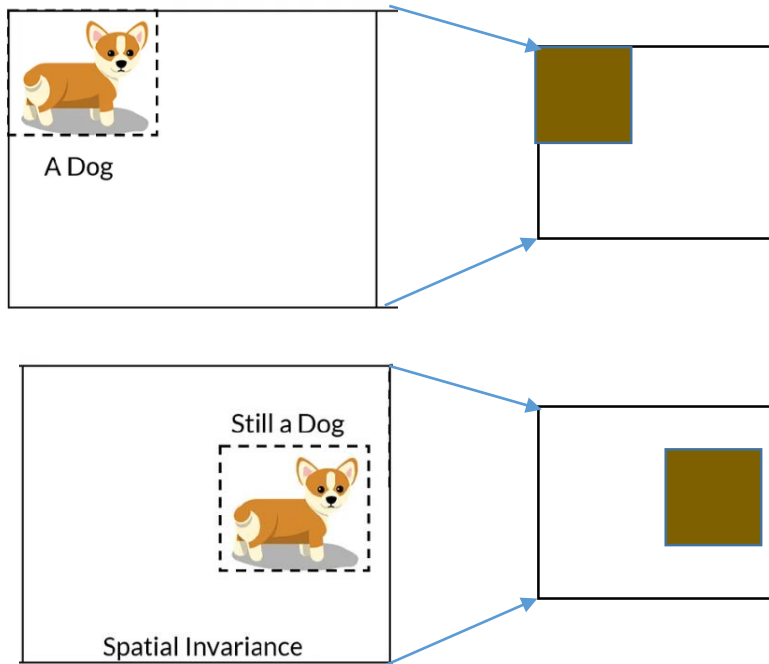
Convolution Neural Networks (CNN)



Convolution Neural Networks (CNN)



Convolution Neural Networks (CNN)



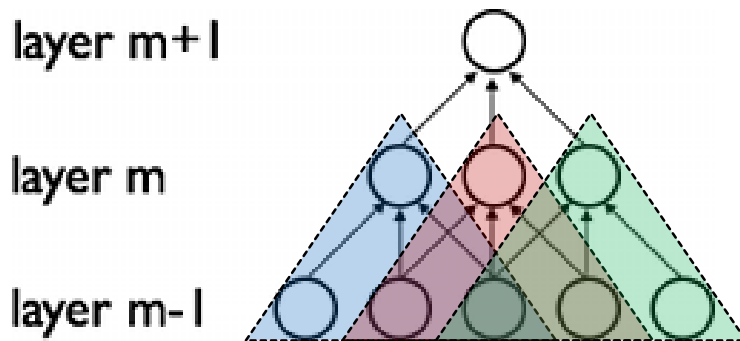
Large Parameter -> Large Data or Overfitting

Convolution Neural Networks (CNN)

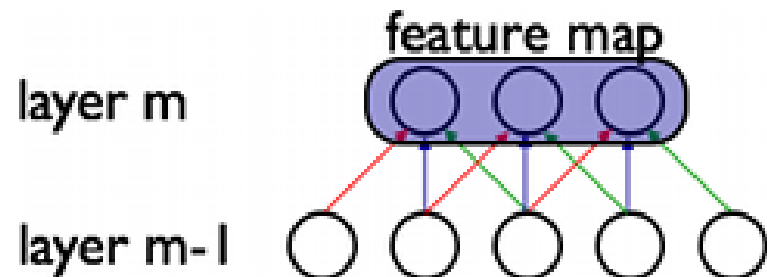
Image, Vision

- **Spatially-local correlation** 을 고려하기 위해 **sparse connection** 구성
 - 인접한 변수만을 이용하여 새로운 feature 생성
- **Invariant feature** 를 추출하기 위해 **shared weight** 개념 이용
 - 인접한 변수 집합에는 동일한 weight를 적용
- 위의 둘을 실현시킨 방법이 **Convolution!**

Sparse Connection



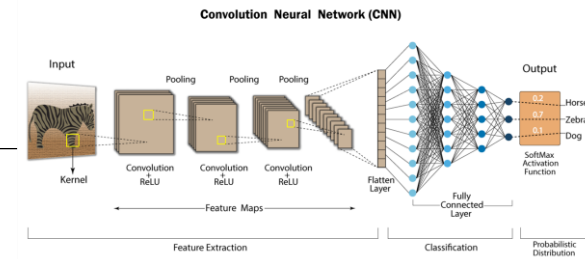
Shared Weight



Convolution Neural Networks (CNN)

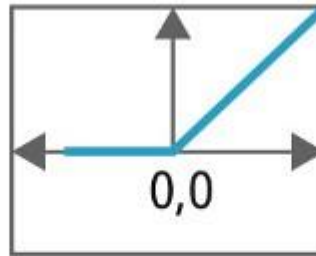
Activation

- Convolution을 통해 학습된 값들의 비선형 변환
- Rectified Linear Unit (ReLU)를 주로 사용



ReLU Activation Function

15	20	-10	35
18	-110	25	100
20	-15	25	-10
101	75	18	23



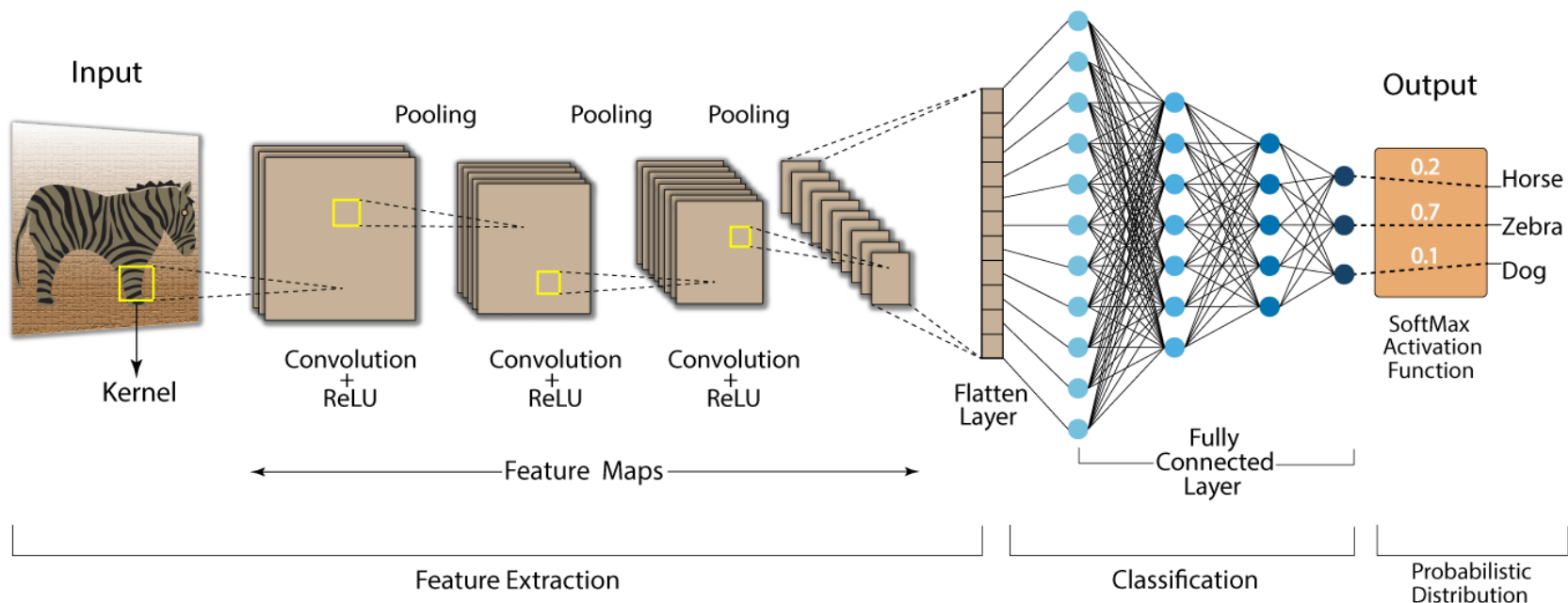
15	20	0	35
18	0	25	100
20	0	25	0
101	75	18	23

Convolution Neural Networks (CNN)

Basic Idea of CNN

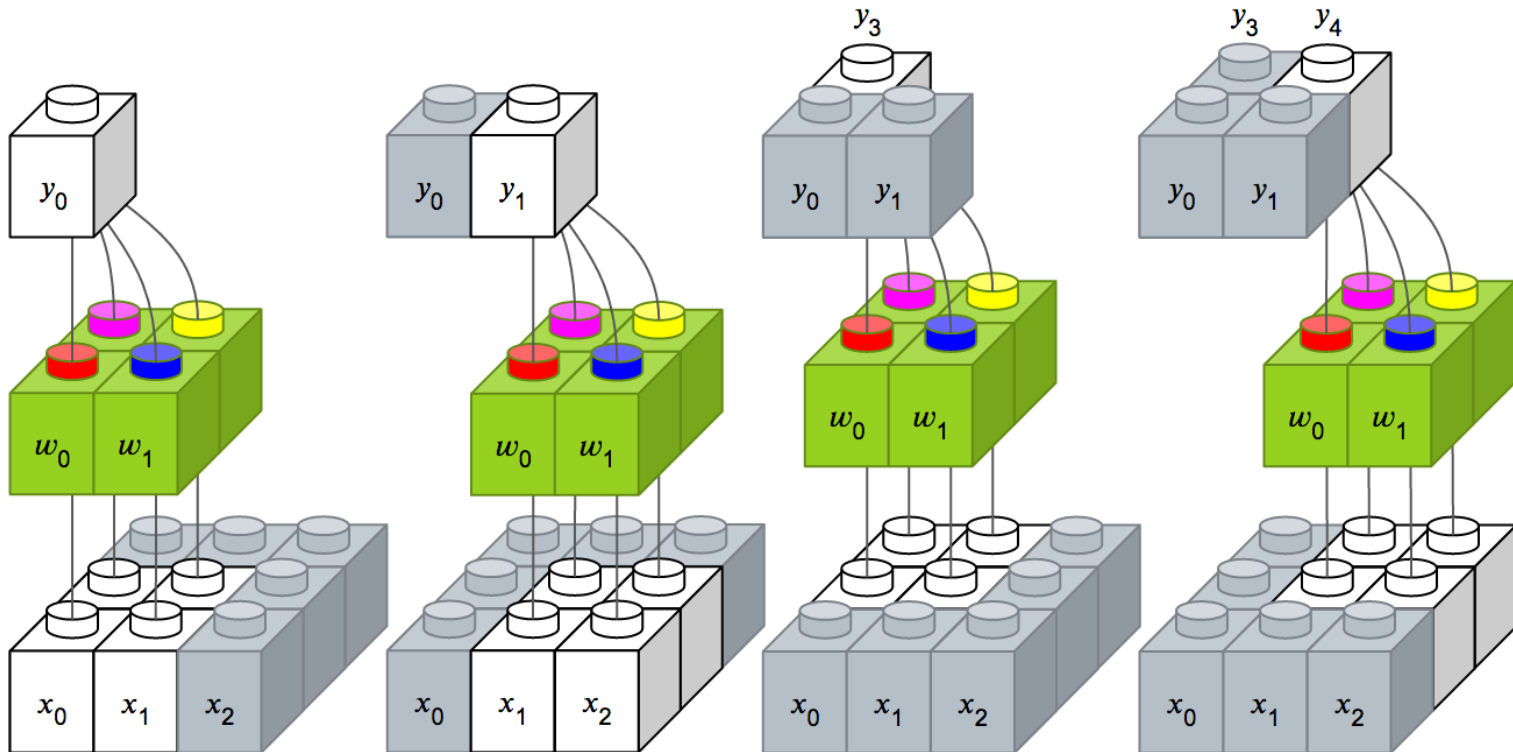
- 이미지 데이터의 특성을 잘 반영할 수 있는 인공신경망 모델
- 2D 혹은 3D구조를 유지하면서 학습
- 일반적인 CNN은 Convolution 연산, Activation 연산, Pooling 연산의 반복으로 구성됨 (Feature Learning, Feature representation)
- 일정 횟수 이상의 Feature Learning 과정 이후에는 Flatten 과정을 통해 이미지가 1차원의 벡터로 변환됨

Convolution Neural Network (CNN)

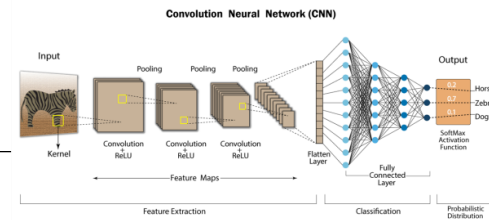


Convolution Neural Networks (CNN)

Basic Idea of CNN



Convolution Neural Networks (CNN)

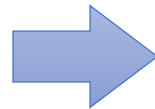


Padding

- Convolution 결과 이미지가 입력 이미지의 크기와 같으면 편리할 때가 많음
- 가장자리에 있는 픽셀들은 중앙에 위치한 픽셀들에 비해 Convolution 연산이 적게 수행됨
- 이를 위한 해결책으로 원 이미지 테두리에 0의 값을 갖는 픽셀 추가
- 테두리에 추가된 0값의 픽셀 → Pad, 테두리에 추가된 0값의 픽셀 추가 → Zero Padding

3	1	1	2	8	4
1	0	7	3	2	6
2	3	5	1	1	3
1	4	1	2	6	5
3	2	1	3	7	2
9	2	6	2	5	1

Padding
with size 1

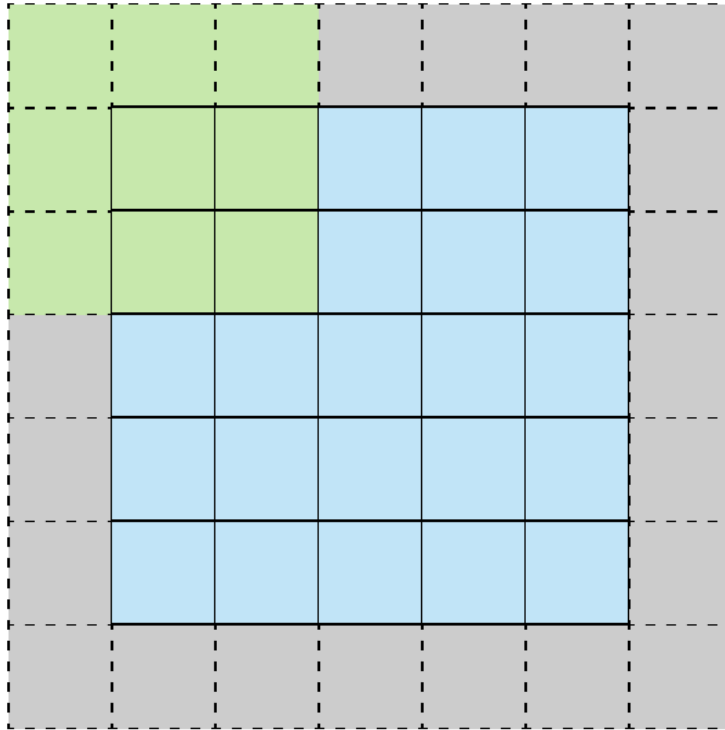


0	0	0	0	0	0	0	0
0	3	1	1	2	8	4	0
0	1	0	7	3	2	6	0
0	2	3	5	1	1	3	0
0	1	4	1	2	6	5	0
0	3	2	1	3	7	2	0
0	9	2	6	2	5	1	0
0	0	0	0	0	0	0	0

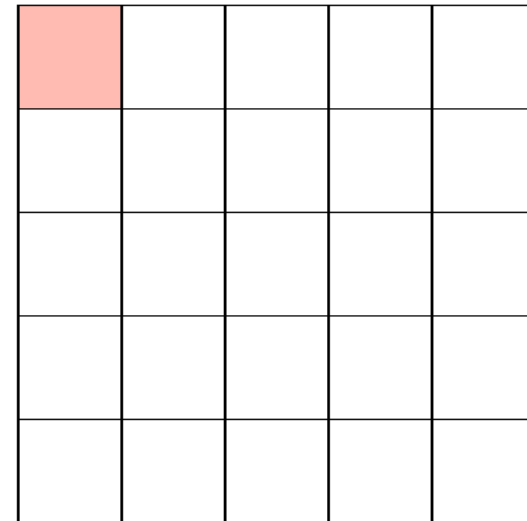
Convolution Neural Networks (CNN)

Padding

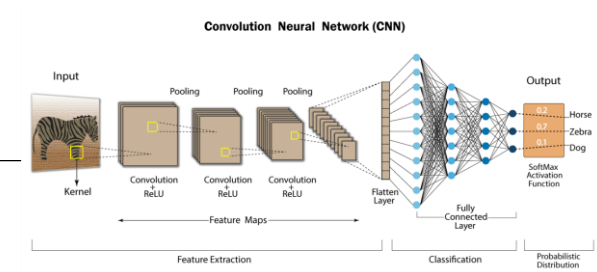
- 원 이미지 테두리에 0값의 픽셀 추가 → Padding (Zero-Padding)



Stride 1 with Padding



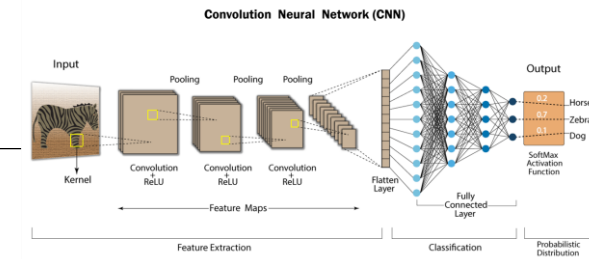
Feature Map



Convolution Neural Networks (CNN)

Padding

- Zero-padding, 1 stride 적용한 합성곱 연산 예제



0	0	0	0	0	0
0	105	102	100	97	96
0	103	99	103	101	102
0	101	98	104	102	100
0	99	101	106	104	99
0	104	104	104	100	98

Image Matrix

Filter

Kernel Matrix

0	-1	0
-1	5	-1
0	-1	0

320				

Output Matrix

$$\begin{aligned}
 &0 * 0 + 0 * -1 + 0 * 0 \\
 &+ 0 * -1 + 105 * 5 + 102 * -1 \\
 &+ 0 * 0 + 103 * -1 + 99 * 0 = 320
 \end{aligned}$$

Convolution with horizontal and vertical strides = 1

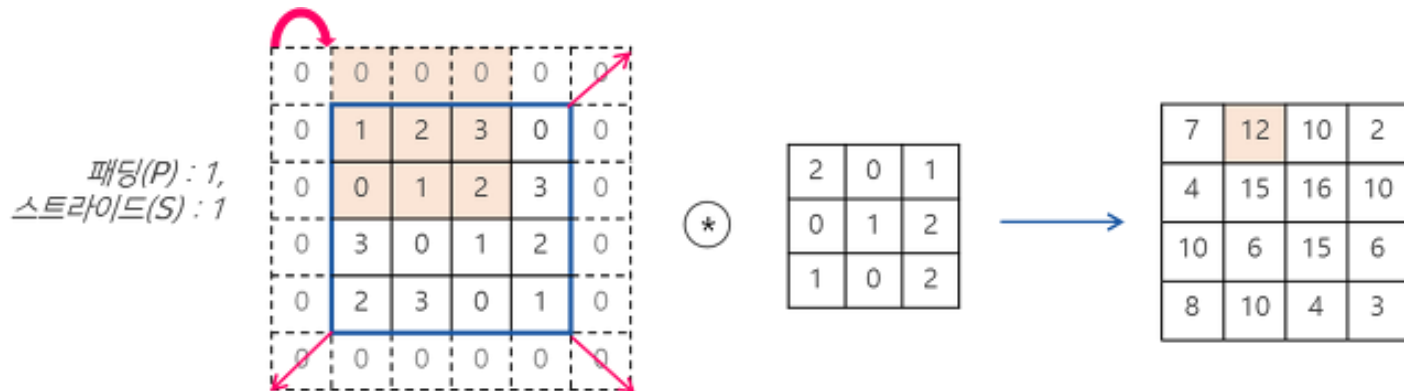
Convolution Neural Networks (CNN)

출력 크기 계산

- 출력 크기 계산

$$(OH, OW) = \left(\frac{H + 2P - FH}{S} + 1, \frac{W + 2P - FW}{S} + 1 \right)$$

- (H, W) : 입력크기
- (FH, FW) : 필터크기
- (OH, OW) : 출력크기
- P : 패딩
- S : 스트라이드



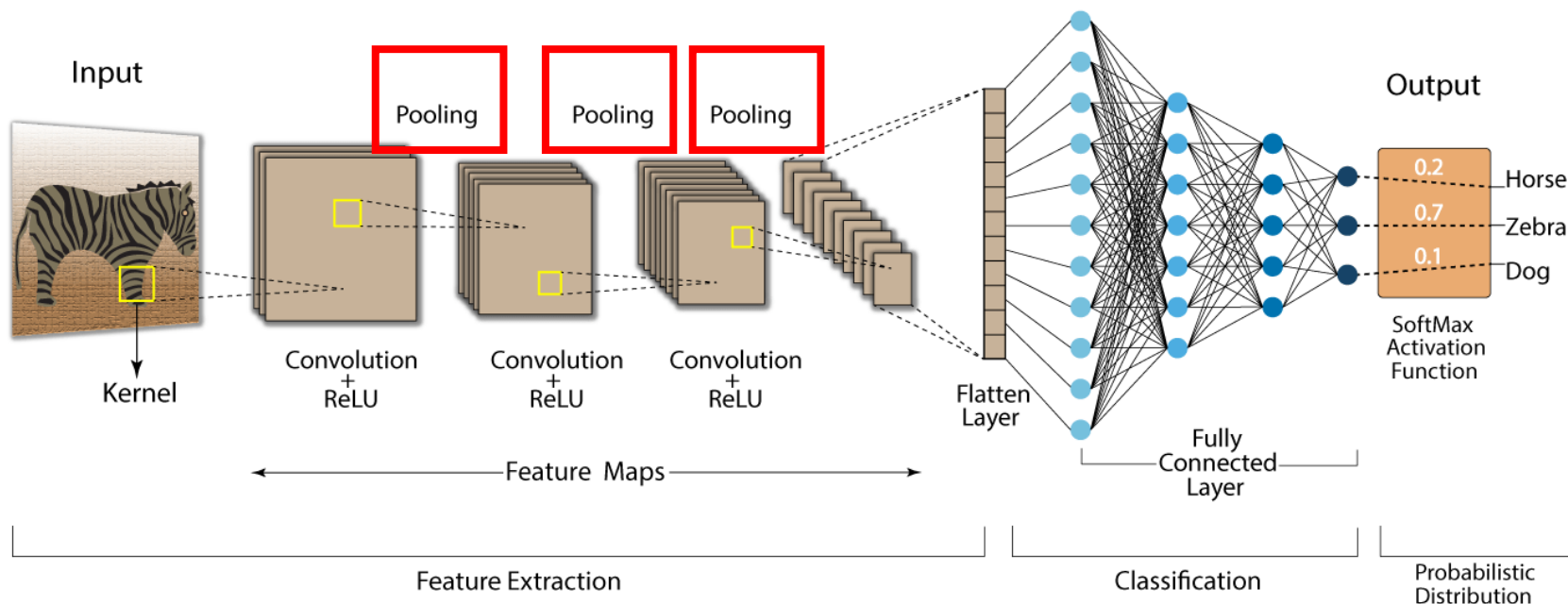
$$(OH, OW) = \left(\frac{4 + 2 * 1 - 3}{1} + 1, \frac{4 + 2 * 1 - 3}{1} + 1 \right) = (4, 4)$$

Convolution Neural Networks (CNN)

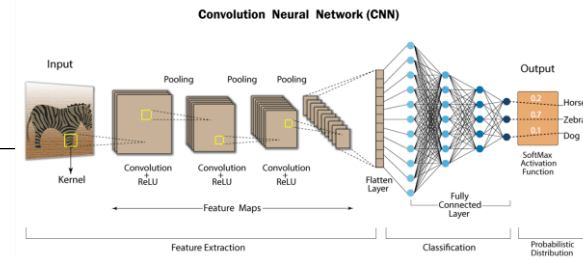
Basic Idea of CNN

- 이미지 데이터의 특성을 잘 반영할 수 있는 인공신경망 모델
- 2D 혹은 3D구조를 유지하면서 학습
- 일반적인 CNN은 Convolution 연산, Activation 연산, Pooling 연산의 반복으로 구성됨 (Feature Learning, Feature representation)
- 일정 횟수 이상의 Feature Learning 과정 이후에는 Flatten 과정을 통해 이미지가 1차원의 벡터로 변환됨

Convolution Neural Network (CNN)



Convolution Neural Networks (CNN)



Pooling

- Feature Map의 공간적 크기를 줄이자
- 일정 영역의 정보를 축약 → Pooling
- Max Pooling, Average Pooling (이미지 분석에서는 주로 Max Pooling)
- Pooling window 크기: 주로 Stride값과 동일하게 설정
- Pooling 층의 유닛에도 활성화 함수를 적용하는 것은 이론적으로 가능 → 보통은 적용 안함
- Pooling 시 padding은 보통 사용하지 않음

Max Pooling

29	15	28	184
0	100	70	38
12	12	7	2
12	12	45	6

2 x 2
pool size

100	184
12	45

Average Pooling

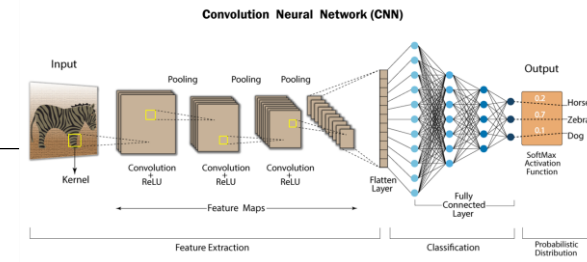
31	15	28	184
0	100	70	38
12	12	7	2
12	12	45	6

2 x 2
pool size

36	80
12	15

Convolution Neural Networks (CNN)

Pooling

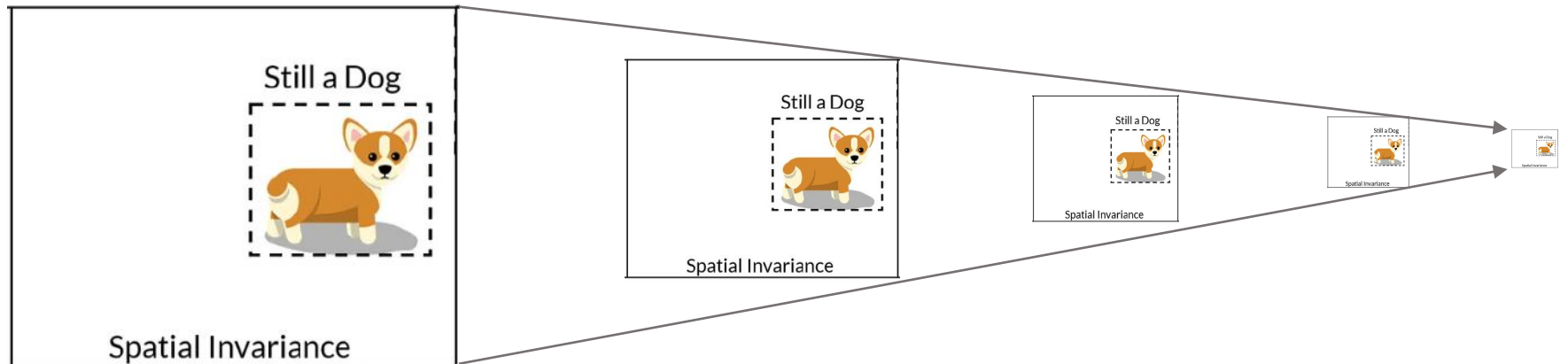
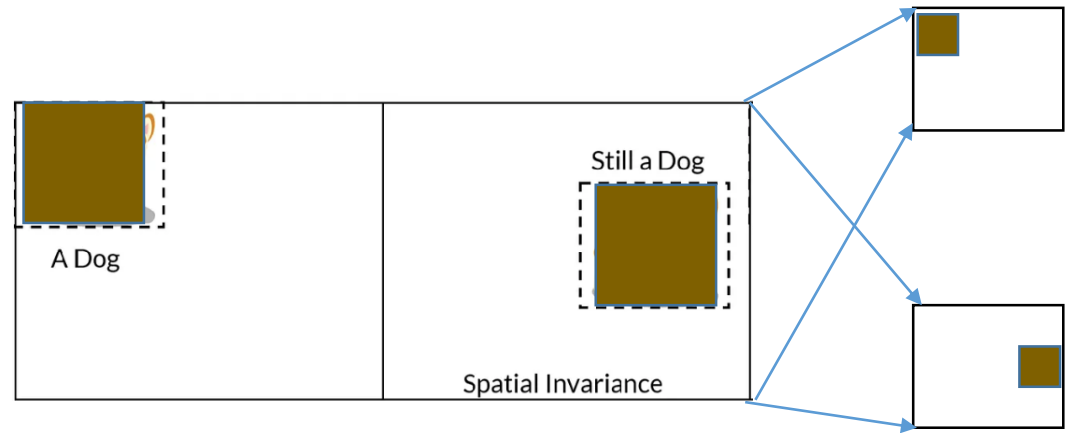


Dimensions reduced
by half

A pooling region always has the same-sized stride as the pool size. This helps to avoid overlapping.

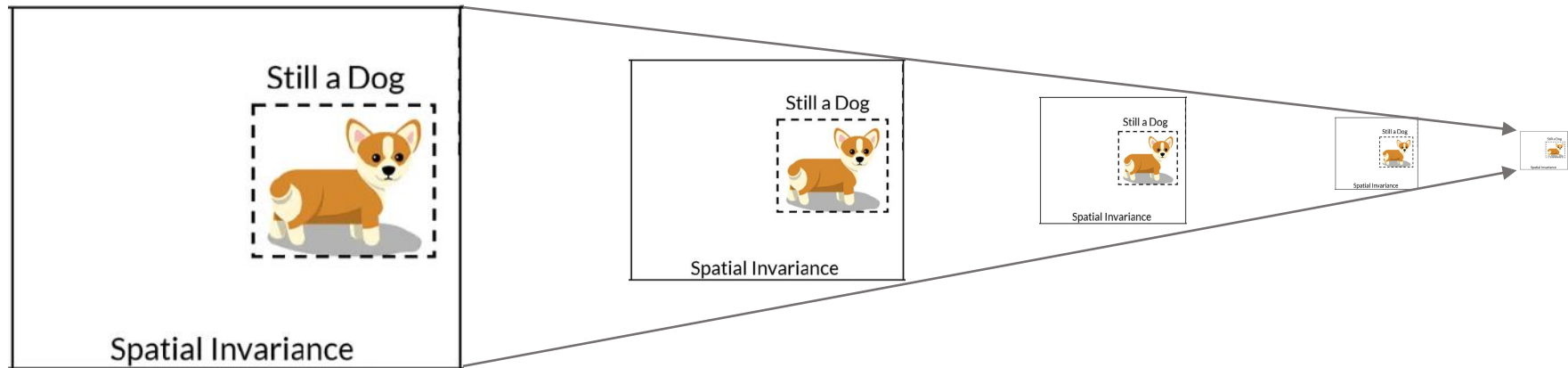
Convolution Neural Networks (CNN)

Pooling



Convolution Neural Networks (CNN)

Pooling



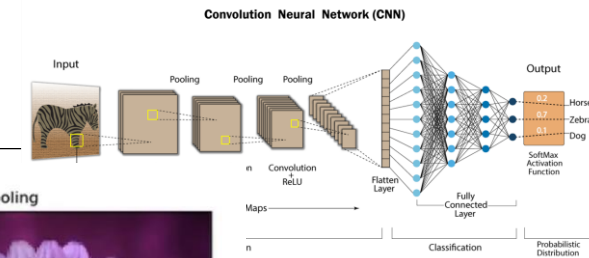
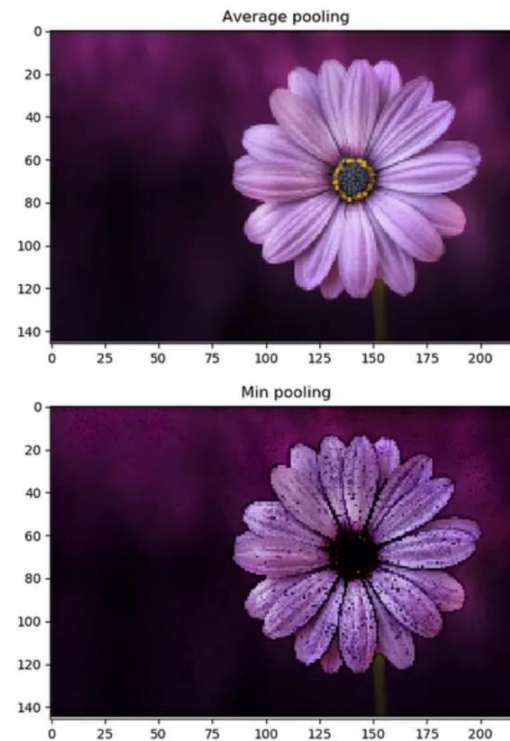
The intuitive reasoning behind this **Pooling layer** is that once we know that a specific feature is in the original input volume (there will be a high activation value), **its exact location is not as important as its relative location to the other features**. As you can imagine, this layer drastically reduces the spatial dimension (the length and the width change but not the depth) of the input volume.

At the deepest layers we have extracted features with no information on where they were positioned on the original image. We even lose the information on pixel-size of original objects because of another process in CNN called pooling.

Convolution is the key for why CNNs perform better than any other model in such “human-like” tasks like recognizing specific objects in the picture, words in a recorded speech and other tasks.

Convolution Neural Networks (CNN)

Pooling



We cannot say that a particular pooling method is better over other generally. The choice of pooling operation is made based on the data at hand. Average pooling method smooths out the image and hence the sharp features may not be identified when this pooling method is used.

Max pooling selects the brighter pixels from the image. It is useful when the background of the image is dark and we are interested in only the lighter pixels of the image. For example: in MNIST dataset, the digits are represented in white color and the background is black. So, max pooling is used. Similarly, min pooling is used in the other way round.

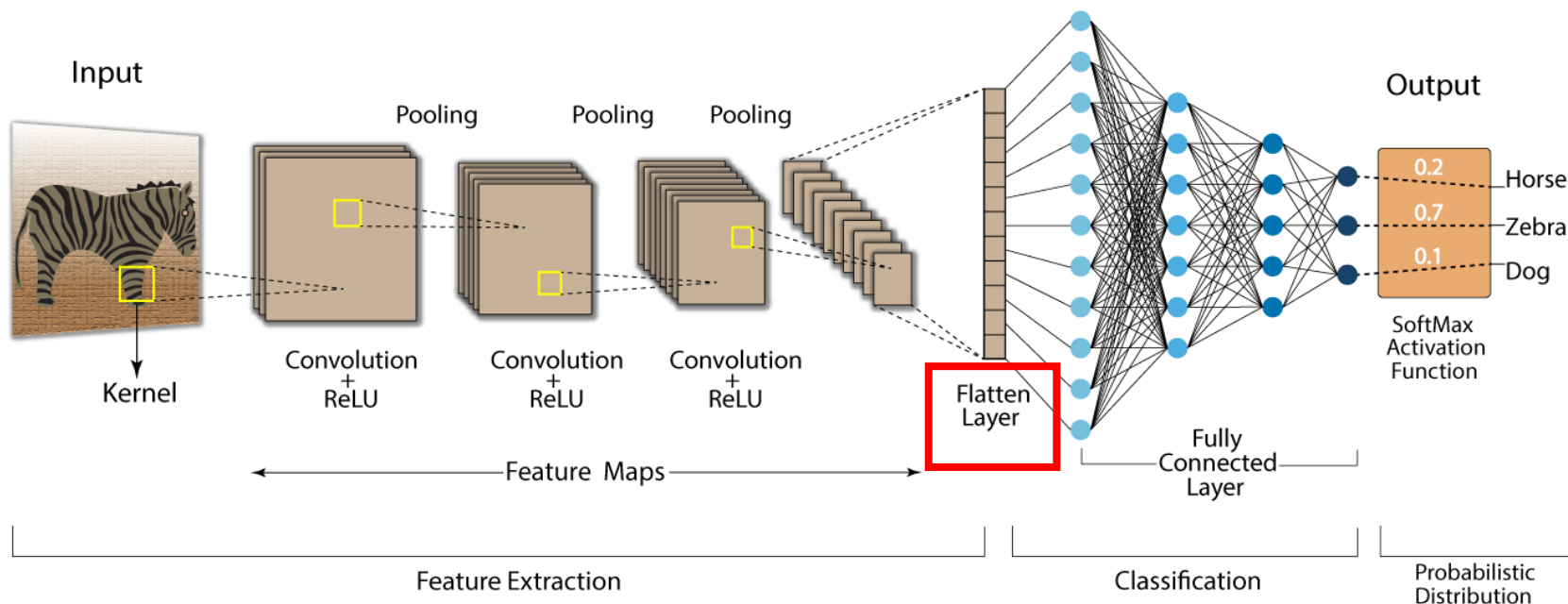
<https://medium.com/@bdhuma/which-pooling-method-is-better-maxpooling-vs-minpooling-vs-average-pooling-95fb03f45a9>

Convolution Neural Networks (CNN)

Basic Idea of CNN

- 이미지 데이터의 특성을 잘 반영할 수 있는 인공신경망 모델
- 2D 혹은 3D구조를 유지하면서 학습
- 일반적인 CNN은 Convolution 연산, Activation 연산, Pooling 연산의 반복으로 구성됨 (Feature Learning, Feature representation)
- 일정 횟수 이상의 Feature Learning 과정 이후에는 Flatten 과정을 통해 이미지가 1차원의 벡터로 변환됨

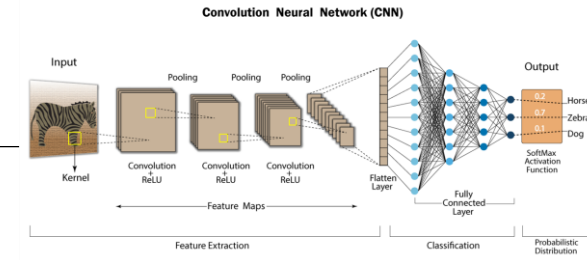
Convolution Neural Network (CNN)



Convolution Neural Networks (CNN)

Flattening

- 2차원, 3차원의 행렬/텐서 구조를 1차원의 벡터로 변환 → Flattening



1	1	0
4	2	1
0	2	1

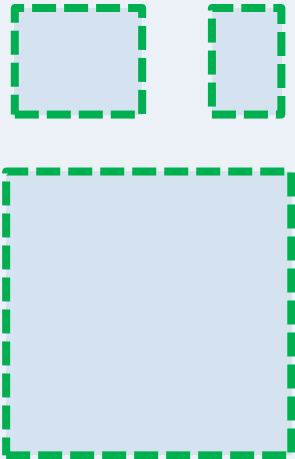
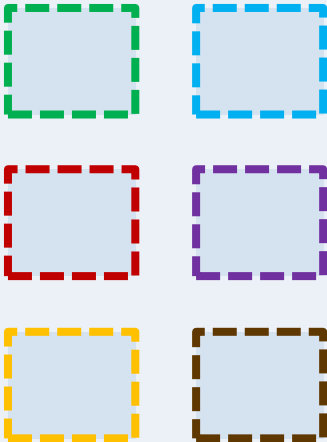
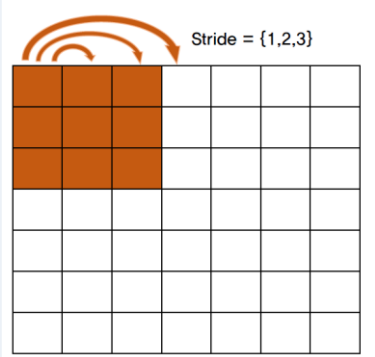
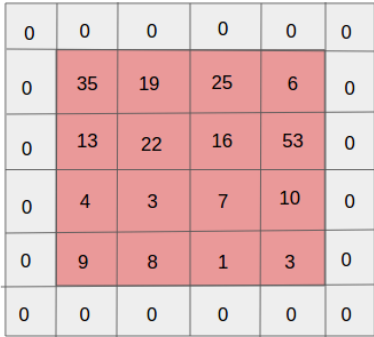
Pooled Feature Map

Flattening

1
1
0
4
2
1
0
2
1

Convolution Neural Networks (CNN)

CNN의 하이퍼파라미터

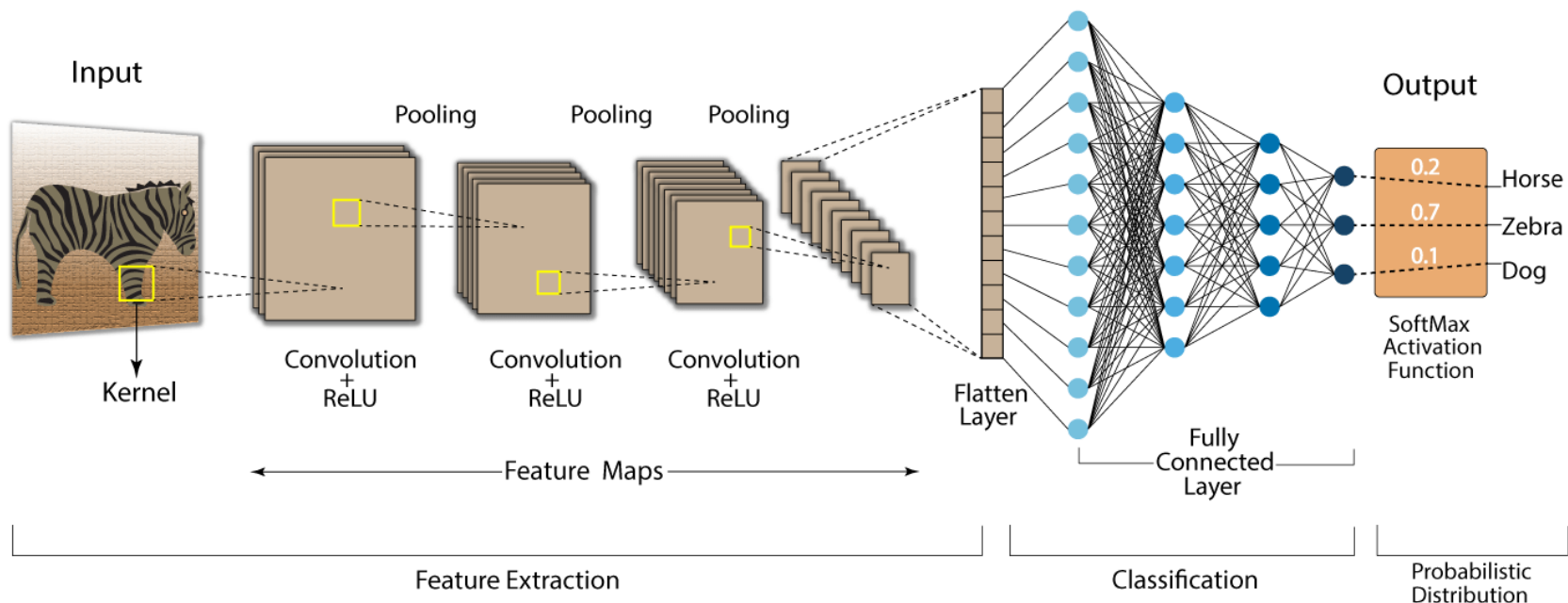
Filter 크기	Filter 수	Stride	Zero Padding
- Filter의 크기와 모양을 결정 (가로×세로)	- 얼마나 다양한 정보를 추출할 것인지 결정 - 많을 수록 다양성 증가	- Filter가 건너뛰는 픽셀의 수 - 클수록 듬성듬성 이동	- Convolution 연산의 편의성을 위해 사용 - Filter 크기 및 수, stride에 따라 결정
			

Convolution Neural Networks (CNN)

Basic Idea of CNN

- 이미지 데이터의 특성을 잘 반영할 수 있는 인공신경망 모델
- 2D 혹은 3D구조를 유지하면서 학습
- 일반적인 CNN은 Convolution 연산, Activation 연산, Pooling 연산의 반복으로 구성됨 (Feature Learning, Feature representation)
- 일정 횟수 이상의 Feature Learning 과정 이후에는 Flatten 과정을 통해 이미지가 1차원의 벡터로 변환됨

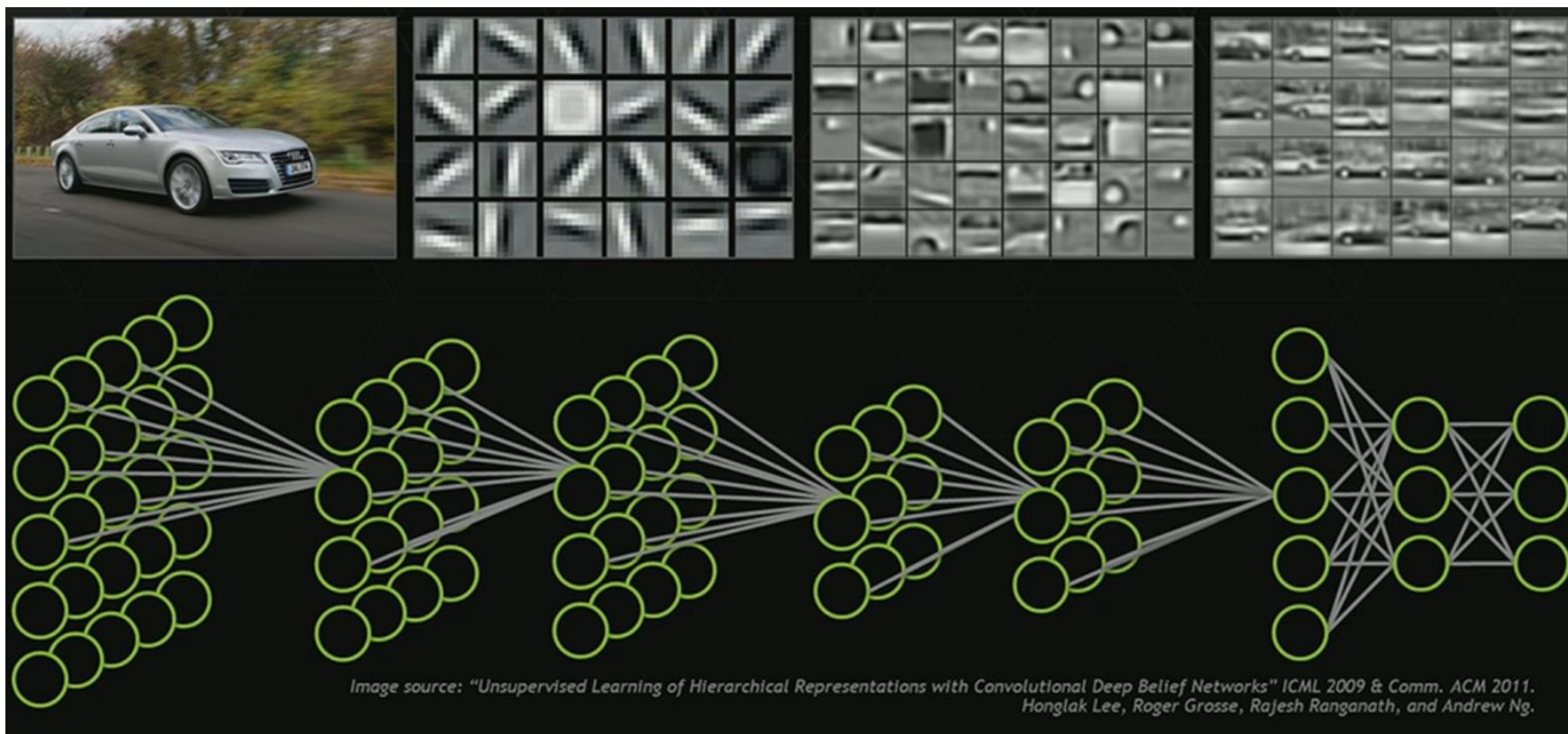
Convolution Neural Network (CNN)



Deep Neural Network (DNN, 심층 신경망)

- Key advantage of DNN

- ✓ 고차원의 원 데이터(이미지, 음성, 텍스트)를 그대로 사용해서도 자동으로 속성(feature)을 정의

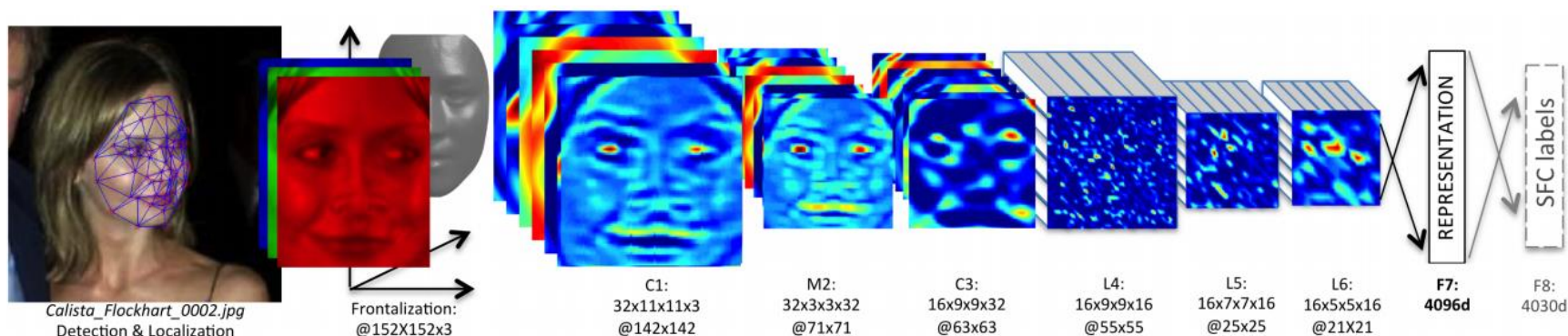


Deep Neural Network (DNN, 심층 신경망)

- Key advantage of DNN

- ✓ 기존의 기계학습 방법론에 비해 우수한 예측 정확도

- 특정 분야(음성, 이미지, 텍스트 인식 등)에서는 인간의 능력과 유사할 정도로 월등히 우수한 성능을 나타내기도 함



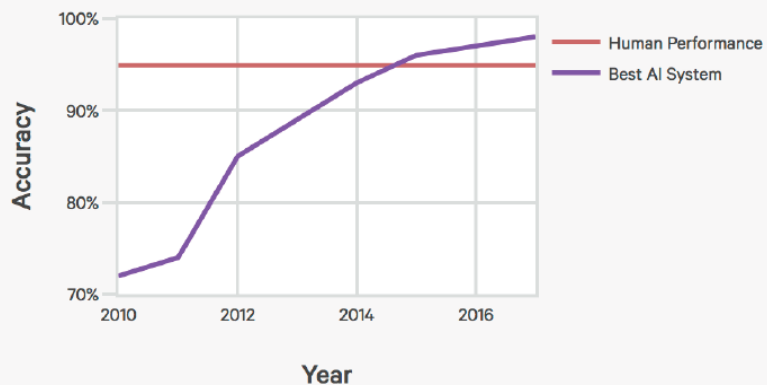
Taigman, Y. et al. (2014). DeepFace: Closing the Gap to Human-Level Performance in Face Verification, CVPR'14.

Recognition Accuracy for Labeled Faces in the Wild (LFW) dataset (13,233 images, 5,749 people)

- Human: 95% vs. DeepFace in Facebook: 97.35%

Deep Neural Network (DNN, 심층 신경망)

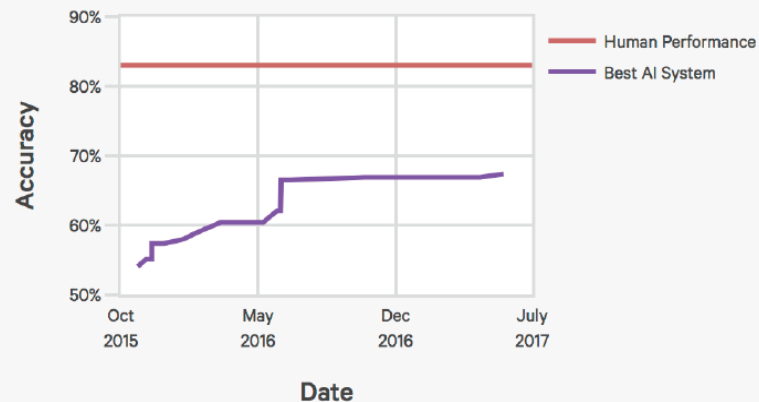
Object Detection, LSVRC Competition



Source: image-net.org

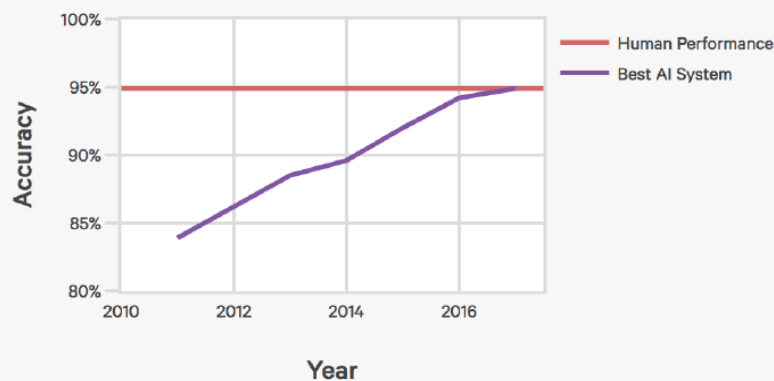
AIINDEX.ORG 

Visual Question Answering, VQA 1.0



AIINDEX.ORG 

Speech Recognition, Switchboard HUB5'00

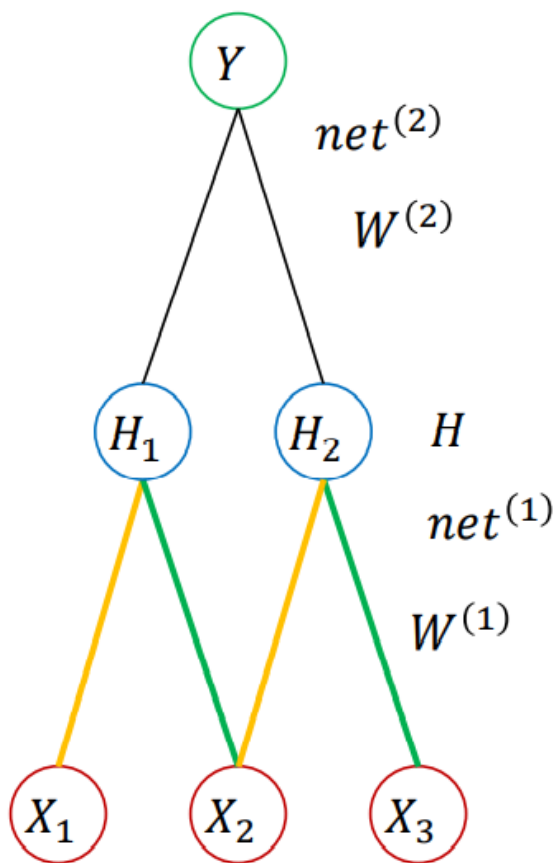


Source: Electronic Frontier Foundation, AI Progress Metrics

AIINDEX.ORG 

Back Propagation Algorithm

- ❖ 틀린 예측을 바로잡기 위해 파라미터를 순차적으로 수정해 나가는 학습 알고리즘
- ❖ net : activation function 이전의 값 (=선형변환)



$$L = y \ln \hat{y}$$

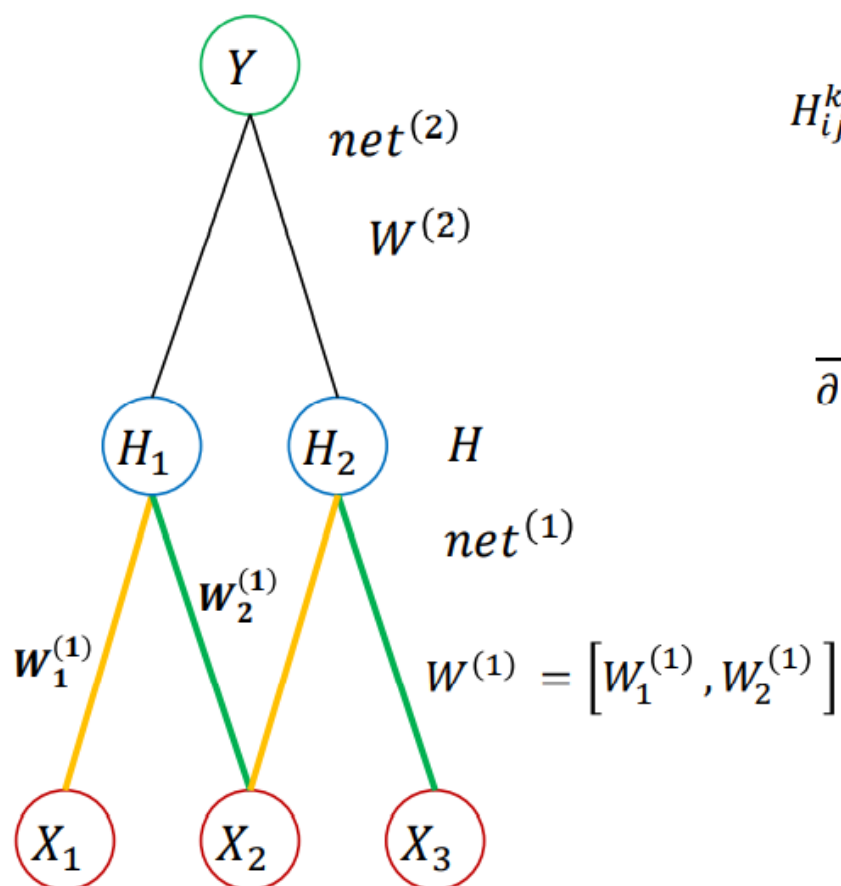
$$\hat{y} = \sigma(net^{(2)}) = \sigma(HW^{(2)})$$

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \sigma} \frac{\partial \sigma}{\partial net^{(2)}} \frac{\partial net^{(2)}}{\partial W^{(2)}}$$

$$W^{(2)} \leftarrow W^{(2)} - \eta \frac{\partial L}{\partial W^{(2)}}$$

Back Propagation Algorithm

- ❖ Convolution layer의 gradient
- ❖ net : activation function 이전의 값 (=선형변환)



$$H_{ij}^k = \text{Activation} \left(\sum_{c \in \text{Channel}} (W_c^k * x_c)_{ij} + b_k \right)$$

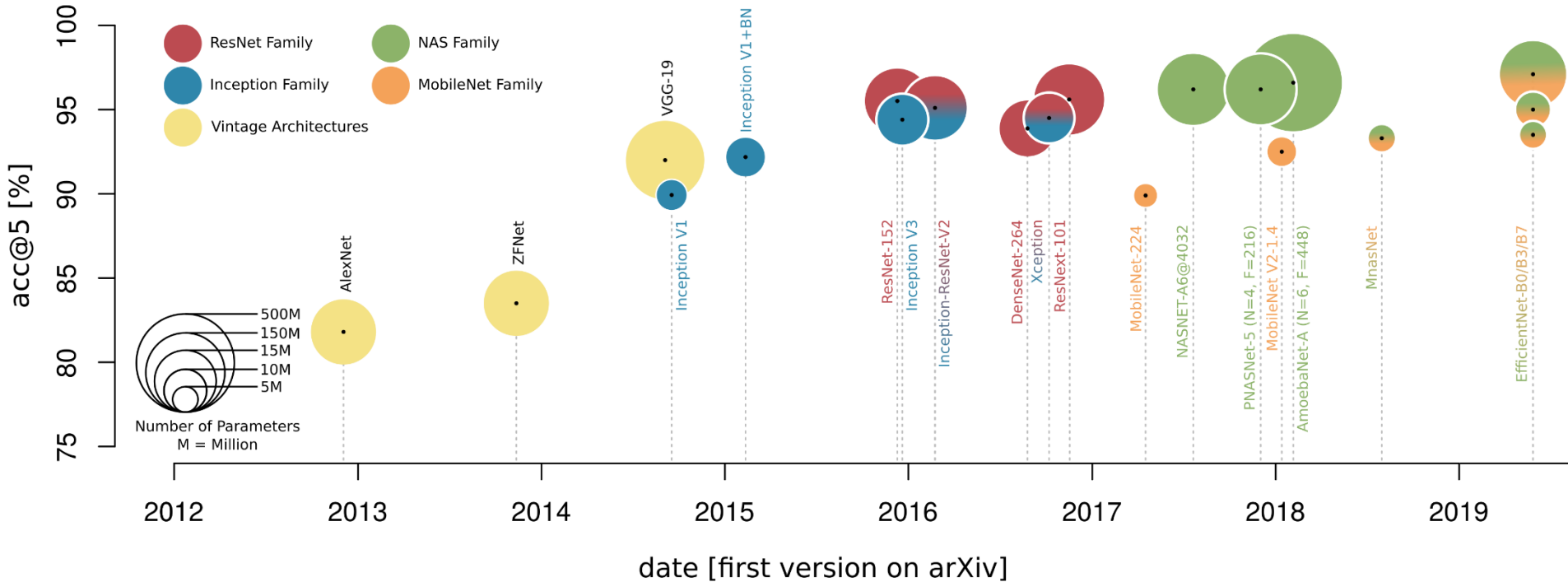
$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \sigma} \frac{\partial \sigma}{\partial net^{(2)}} \frac{\partial net^{(2)}}{\partial H} \frac{\partial H}{\partial \sigma} \frac{\partial \sigma}{\partial net^{(1)}} \frac{\partial net^{(1)}}{\partial W^{(1)}}$$

$$\frac{\partial net^{(1)}}{\partial W_i^{(1)}} = \left(\frac{\partial net_1^{(1)}}{\partial W_i^{(1)}} + \frac{\partial net_2^{(1)}}{\partial W_i^{(1)}} \right)$$

$$W^{(1)} \leftarrow W^{(1)} - \eta \frac{\partial L}{\partial W^{(1)}}$$

Convolution Neural Networks (CNN)

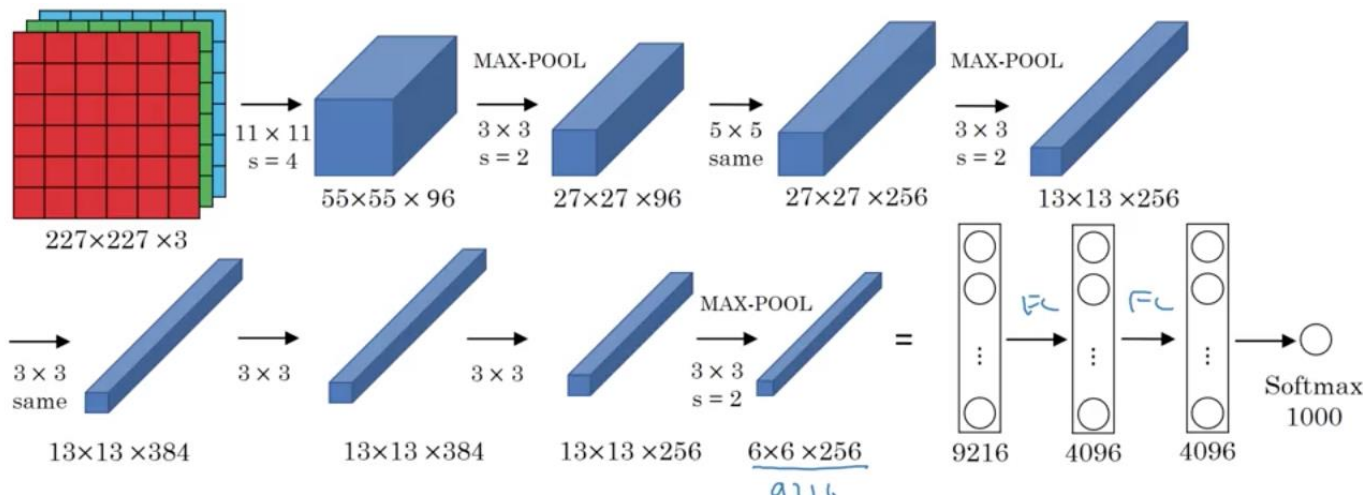
CNN 구조 (Architecture of CNN)



[Source] Hoer, T; Kuenzer, C. Object Detection and Image Segmentation with Deep Learning on Earth Observation Data: A Review-Part I: Evolution and Recent Trends. Remote Sensing 2020, 12(10), 1667.

Convolution Neural Networks (CNN)

AlexNet



[227x227x3] Input

[55x55x96] **Conv1**: 96 11x11 filters at stride 4, pad 0 (96개의 다른 종류 11x11 filter 사용)

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization Layer

[27x27x256] **Conv2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization Layer

[13x13x384] **Conv3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **Conv4**: 384 3x3 filters at stride 1, pad 1

[13x13x256] **Conv5**: 256 3x3 filters at stride 1, pad 1

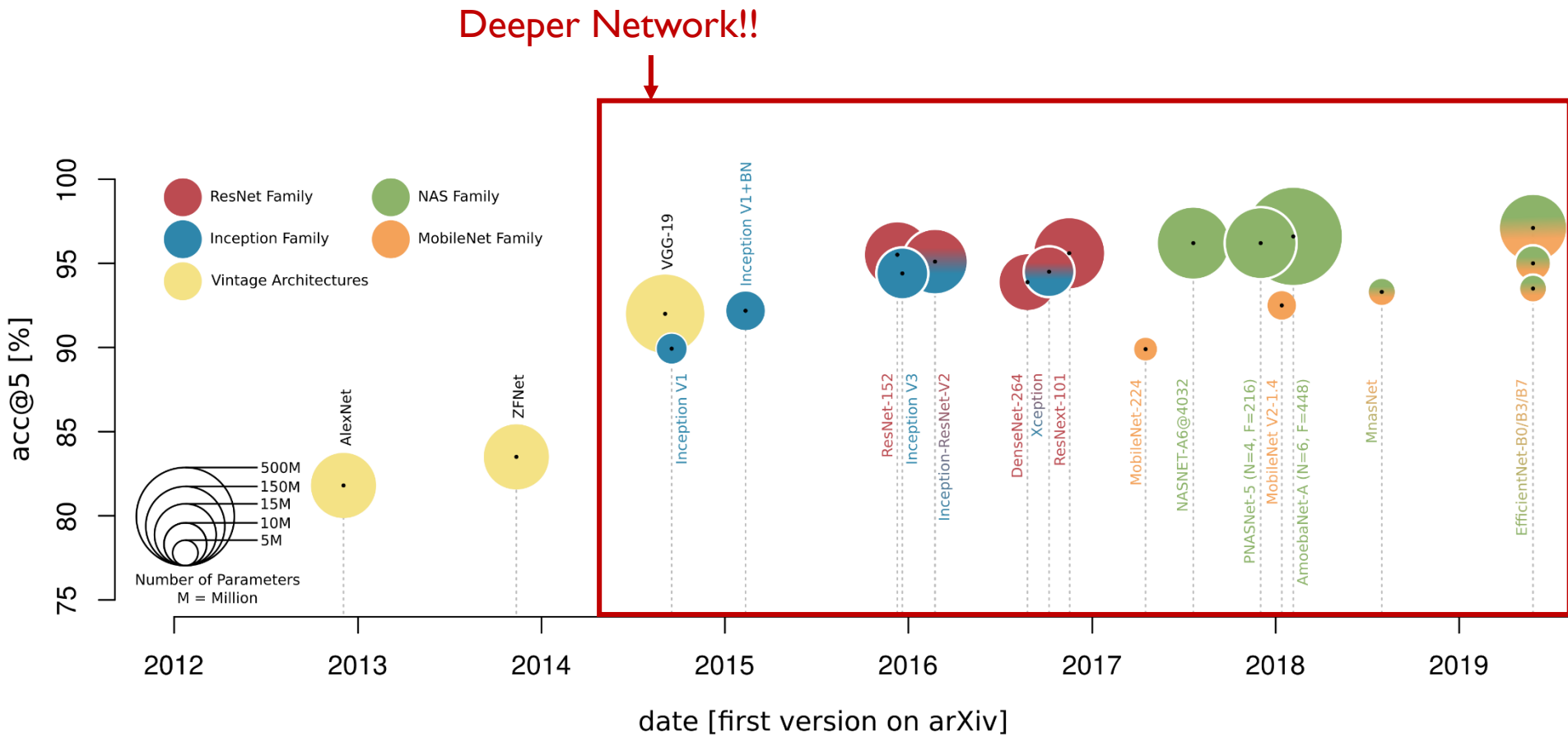
[6 x 6 x 256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)

Convolution Neural Networks (CNN)



EOD